

Pointer-Based String Processing

- Character constant
 - Enclosed in single quotes,
 - forexample: 'z'
- A string is a series of characters treated as a single unit.
 - May include letters, digits and various **special characters** such as +, -, *, / and \$.
- A string is an array of characters ending with a **null character** ('\0')
- String
 - Enclosed in double quotes,
 - forexample: "I like C++"

Pointer-Based String Processing

- All strings end with null ('\\0')
- Character array
 - Creates 5 element **char** array **color**
 - Null character ('\\0') implicitly added
- Alternative for character array

```
char color[ 5 ] = { 'b', 'l', 'u', 'e', '\\0' };
```

	0	1	2	3	4
color	b	l	u	e	\\0

color[0]	b	0012FF48
color[1]	l	0012FF49
color[2]	u	0012FF4A
color[3]	e	0012FF4B
color[4]	\\0	0012FF4D

Pointer-Based String Processing

- All strings end with null ('\\0')
- Character array
 - Creates 5 element **char** array **color**
 - Null character ('\\0') implicitly added
- Alternative for character array

```
char color[] = { 'b', 'l', 'u', 'e', '\\0' };
```

	0	1	2	3	4
color	b	l	u	e	\\0

color[0]	b	0012FF48
color[1]	l	0012FF49
color[2]	u	0012FF4A
color[3]	e	0012FF4B
color[4]	\\0	0012FF4D

Pointer-Based String Processing

- Input from keyboard

```
char color[ 10 ];  
cin >> color;
```

- Puts user input in string
 - Stops at first whitespace character
 - Adds **null** character
- If too much text entered, run time error

- Printing strings

```
cout << color << endl;
```

- Does not work for other array types
- Characters printed until **null** found

```
int main()
{
    char string1[ 20 ];
    char string2[] = "string literal";

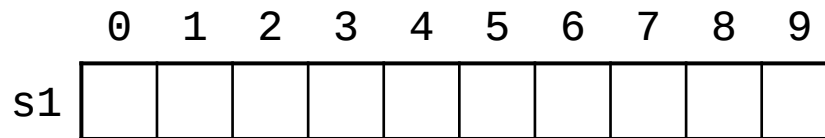
    cout << "Enter the string \"hello there\": ";
    cin >> string1; // reads "hello"

    cout << "string1 is: " << string1 << "\nstring2 is: " << string2;

    cout << "\nstring1 with spaces between characters is:\n";
    for( int i = 0; string1[ i ] != '\0'; i++ )
        cout << string1[ i ] << ' ';

    cin >> string1; // reads "there"
    cout << "\nstring1 is: " << string1 << endl;
}
```

```
Enter the string "hello there": hello there
string1 is: hello
string2 is: string literal
string1 with spaces between characters is:
h e l l o
string1 is: there
```



```

int main()
{
    char s1[ 10 ];

    cin >> s1; // reads "hello"

    cout << "string1 is: " << s1;

    for( int i = 0; s1[ i ] != '\0'; i++ )
        cout << s1[ i ] << ' ';

    cin >> s1; // reads "there"
    cout << "\nstring1 is: " << s1 << endl;
}

```

s1[0]		0012FF48
s1[1]		0012FF49
s1[2]		0012FF4A
s1[3]		0012FF4B
s1[4]		0012FF4C
s1[5]		0012FF4D
s1[6]		0012FF4E
s1[7]		0012FF4F
s1[8]		0012FF50
s1[9]		0012FF51

	0	1	2	3	4	5	6	7	8	9
s1	h	e	l	l	o	\0				

```

int main()
{
    char s1[ 10 ];

    cin >> s1; // reads "hello"

    cout << "string1 is: " << s1;

    for( int i = 0; s1[ i ] != '\0'; i++ )
        cout << s1[ i ] << ' ';

    cin >> s1; // reads "there"
    cout << "\nstring1 is: " << s1 << endl;
}

```

s1[0]	h	0012FF48
s1[1]	e	0012FF49
s1[2]	l	0012FF4A
s1[3]	l	0012FF4B
s1[4]	o	0012FF4C
s1[5]	\0	0012FF4D
s1[6]		0012FF4E
s1[7]		0012FF4F
s1[8]		0012FF50
s1[9]		0012FF51

	0	1	2	3	4	5	6	7	8	9
s1	t	h	e	r	e	\0				

```

int main()
{
    char s1[ 10 ];

    cin >> s1; // reads "hello"

    cout << "string1 is: " << s1;

    for( int i = 0; s1[ i ] != '\0'; i++ )
        cout << s1[ i ] << ' ';

    cin >> s1; // reads "there"
    cout << "\nstring1 is: " << s1 << endl;
}

```

s1[0]	t	0012FF48
s1[1]	h	0012FF49
s1[2]	e	0012FF4A
s1[3]	r	0012FF4B
s1[4]	e	0012FF4C
s1[5]	\0	0012FF4D
s1[6]		0012FF4E
s1[7]		0012FF4F
s1[8]		0012FF50
s1[9]		0012FF51

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
s2	s	t	r	i	n	g		l	i	t	e	r	a	l	\0

```

int main()
{
    char s2[] = "string literal";

    cout << "string2 is: " << s2;
}

```

s1[0]	s	0012FF48
s1[1]	t	0012FF49
s1[2]	r	0012FF4A
s1[3]	i	0012FF4B
s1[4]	n	0012FF4C
s1[5]	g	0012FF4D
s1[6]		0012FF4E
s1[7]	l	0012FF4F
s1[8]	i	0012FF50
s1[9]	t	0012FF51
s1[10]	e	0012FF52
s1[11]	r	0012FF53
s1[12]	a	0012FF54
s1[13]	l	0012FF55
s1[14]	\0	0012FF56

```

#include <iostream>
#include <cctype>
using namespace std;

void convertToUppercase( char * );

int main()
{
    char phrase[] = "characters and $32.98";

    cout << "The phrase before conversion is: " << phrase;
    convertToUppercase( phrase );
    cout << "\nThe phrase after conversion is: " << phrase << endl;
}

void convertToUppercase( char *sPtr )
{
    for( ; *sPtr != '\0'; sPtr++ )
        if( islower( *sPtr ) )
            *sPtr = toupper( *sPtr );
}

```

The phrase before conversion is: characters and \$32.98

The phrase after conversion is: CHARACTERS AND \$32.98

```

int main()
{
    char phrase[] = "characters and $32.98";
    convertToUppercase( phrase );
}

void convertToUppercase( char *sPtr )
{
    for( ; *sPtr != '\0'; ++sPtr )
        if ( islower( *sPtr ) )
            *sPtr = toupper( *sPtr );
}

```

phrase[0]	c	0012FF48
phrase[1]	h	0012FF49
phrase[2]	a	0012FF4A
phrase[3]	r	0012FF4B
phrase[4]	a	0012FF4C
phrase[5]	c	0012FF4D
phrase[6]	t	0012FF4E
phrase[7]	e	0012FF4F
phrase[8]	r	0012FF50
phrase[9]	s	0012FF51
phrase[10]		0012FF52
phrase[11]	a	0012FF53
phrase[12]	n	0012FF54
phrase[13]	d	0012FF55
phrase[14]		0012FF56
phrase[15]	\$	0012FF57
phrase[16]	3	0012FF58
phrase[17]	2	0012FF59

sPtr 0012FF48 0012FE74

```
int main()
{
    char phrase[] = "characters and $32.98";
    convertToUppercase( phrase );
}

void convertToUppercase( char *sPtr )
{
    for( ; *sPtr != '\0'; ++sPtr )
        if ( islower( *sPtr ) )
            *sPtr = toupper( *sPtr );
}
```

phrase[0]	c	0012FF48
phrase[1]	h	0012FF49
phrase[2]	a	0012FF4A
phrase[3]	r	0012FF4B
phrase[4]	a	0012FF4C
phrase[5]	c	0012FF4D
phrase[6]	t	0012FF4E
phrase[7]	e	0012FF4F
phrase[8]	r	0012FF50
phrase[9]	s	0012FF51
phrase[10]		0012FF52
phrase[11]	a	0012FF53
phrase[12]	n	0012FF54
phrase[13]	d	0012FF55
phrase[14]		0012FF56
phrase[15]	\$	0012FF57
phrase[16]	3	0012FF58
phrase[17]	2	0012FF59

sPtr 0012FF48 0012FE74

```
int main()
{
    char phrase[] = "characters and $32.98";
    convertToUppercase( phrase );
}

void convertToUppercase( char *sPtr )
{
    for( ; *sPtr != '\0'; ++sPtr )
        if ( islower( *sPtr ) )
            *sPtr = toupper( *sPtr );
}
```

phrase[0]	c	0012FF48
phrase[1]	h	0012FF49
phrase[2]	a	0012FF4A
phrase[3]	r	0012FF4B
phrase[4]	a	0012FF4C
phrase[5]	c	0012FF4D
phrase[6]	t	0012FF4E
phrase[7]	e	0012FF4F
phrase[8]	r	0012FF50
phrase[9]	s	0012FF51
phrase[10]		0012FF52
phrase[11]	a	0012FF53
phrase[12]	n	0012FF54
phrase[13]	d	0012FF55
phrase[14]		0012FF56
phrase[15]	\$	0012FF57
phrase[16]	3	0012FF58
phrase[17]	2	0012FF59

sPtr 0012FF49 0012FE74

```
int main()
{
    char phrase[] = "characters and $32.98";
    convertToUppercase( phrase );
}

void convertToUppercase( char *sPtr )
{
    for( ; *sPtr != '\0'; ++sPtr )
        if ( islower( *sPtr ) )
            *sPtr = toupper( *sPtr );
}
```

phrase[0]	c	0012FF48
phrase[1]	h	0012FF49
phrase[2]	a	0012FF4A
phrase[3]	r	0012FF4B
phrase[4]	a	0012FF4C
phrase[5]	c	0012FF4D
phrase[6]	t	0012FF4E
phrase[7]	e	0012FF4F
phrase[8]	r	0012FF50
phrase[9]	s	0012FF51
phrase[10]		0012FF52
phrase[11]	a	0012FF53
phrase[12]	n	0012FF54
phrase[13]	d	0012FF55
phrase[14]		0012FF56
phrase[15]	\$	0012FF57
phrase[16]	3	0012FF58
phrase[17]	2	0012FF59

sPtr 0012FF49 0012FE74

```
int main()
{
    char phrase[] = "characters and $32.98";
    convertToUppercase( phrase );
}

void convertToUppercase( char *sPtr )
{
    for( ; *sPtr != '\0'; ++sPtr )
        if ( islower( *sPtr ) )
            *sPtr = toupper( *sPtr );
}
```

phrase[0]	c	0012FF48
phrase[1]	h	0012FF49
phrase[2]	a	0012FF4A
phrase[3]	r	0012FF4B
phrase[4]	a	0012FF4C
phrase[5]	c	0012FF4D
phrase[6]	t	0012FF4E
phrase[7]	e	0012FF4F
phrase[8]	r	0012FF50
phrase[9]	s	0012FF51
phrase[10]		0012FF52
phrase[11]	a	0012FF53
phrase[12]	n	0012FF54
phrase[13]	d	0012FF55
phrase[14]		0012FF56
phrase[15]	\$	0012FF57
phrase[16]	3	0012FF58
phrase[17]	2	0012FF59

sPtr 0012FF4A ~~0012FE74~~

```
int main()
{
    char phrase[] = "characters and $32.98";
    convertToUppercase( phrase );
}

void convertToUppercase( char *sPtr )
{
    for( ; *sPtr != '\0'; ++sPtr )
        if ( islower( *sPtr ) )
            *sPtr = toupper( *sPtr );
}
```

phrase[0]	c	0012FF48
phrase[1]	h	0012FF49
phrase[2]	a	0012FF4A
phrase[3]	r	0012FF4B
phrase[4]	a	0012FF4C
phrase[5]	c	0012FF4D
phrase[6]	t	0012FF4E
phrase[7]	e	0012FF4F
phrase[8]	r	0012FF50
phrase[9]	s	0012FF51
phrase[10]		0012FF52
phrase[11]	a	0012FF53
phrase[12]	n	0012FF54
phrase[13]	d	0012FF55
phrase[14]		0012FF56
phrase[15]	\$	0012FF57
phrase[16]	3	0012FF58
phrase[17]	2	0012FF59

sPtr 0012FF4A ~~0012FE74~~

```
int main()
{
    char phrase[] = "characters and $32.98";
    convertToUppercase( phrase );
}

void convertToUppercase( char *sPtr )
{
    for( ; *sPtr != '\0'; ++sPtr )
        if ( islower( *sPtr ) )
            *sPtr = toupper( *sPtr );
}
```

phrase[0]	c	0012FF48
phrase[1]	h	0012FF49
phrase[2]	a	0012FF4A
phrase[3]	r	0012FF4B
phrase[4]	a	0012FF4C
phrase[5]	c	0012FF4D
phrase[6]	t	0012FF4E
phrase[7]	e	0012FF4F
phrase[8]	r	0012FF50
phrase[9]	s	0012FF51
phrase[10]		0012FF52
phrase[11]	a	0012FF53
phrase[12]	n	0012FF54
phrase[13]	d	0012FF55
phrase[14]		0012FF56
phrase[15]	\$	0012FF57
phrase[16]	3	0012FF58
phrase[17]	2	0012FF59

sPtr 0012FF4B ~~0012FE74~~

```
int main()
{
    char phrase[] = "characters and $32.98";
    convertToUppercase( phrase );
}

void convertToUppercase( char *sPtr )
{
    for( ; *sPtr != '\0'; ++sPtr )
        if ( islower( *sPtr ) )
            *sPtr = toupper( *sPtr );
}
```

phrase[0]	c	0012FF48
phrase[1]	h	0012FF49
phrase[2]	a	0012FF4A
phrase[3]	r	0012FF4B
phrase[4]	a	0012FF4C
phrase[5]	c	0012FF4D
phrase[6]	t	0012FF4E
phrase[7]	e	0012FF4F
phrase[8]	r	0012FF50
phrase[9]	s	0012FF51
phrase[10]		0012FF52
phrase[11]	a	0012FF53
phrase[12]	n	0012FF54
phrase[13]	d	0012FF55
phrase[14]		0012FF56
phrase[15]	\$	0012FF57
phrase[16]	3	0012FF58
phrase[17]	2	0012FF59

```
int main()
{
    char phrase[] = "characters and $32.98";

    cout << "The phrase before conversion is: " << phrase;
    convertToUppercase( phrase );
    cout << "\nThe phrase after conversion is: " << phrase << endl;
}
```

```
int main()
{
    string phrase( "characters and $32.98" );

    cout << "The phrase before conversion is: " << phrase;
    convertToUppercase( phrase );
    cout << "\nThe phrase after conversion is: " << phrase << endl;
}
```

```
void convertToUppercase( char *s )
{
    for( int i = 0; i < strlen( s ); i++ )
        if( islower( s[i] ) )
            s[i] = toupper( s[i] );
}
```

```
void convertToUppercase( string s )
{
    for( int i = 0; i < s.length(); i++ )
        if( islower( s[i] ) )
            s[i] = toupper( s[i] );
}
```

```
#include <iostream>
using namespace std;

void printCharacters( const char * );

int main()
{
    const char phrase[] = "print characters of a string";

    cout << "The string is:\n";
    printCharacters( phrase );
    cout << endl;
}

void printCharacters( const char *sPtr )
{
    for( ; *sPtr != '\0'; sPtr++ )
        cout << *sPtr;
}
```

The string is:
print characters of a string


```

int main()
{
    const char phrase[] =
        "print characters of a string";
    printChar( phrase );
}

void printChar( const char *sPtr )
{
    for( ; *sPtr != '\0'; sPtr++ )
        cout << *sPtr;
}

```

phrase[0]	p	0012FF48
phrase[1]	r	0012FF49
phrase[2]	i	0012FF4A
phrase[3]	n	0012FF4B
phrase[4]	t	0012FF4C
phrase[5]		0012FF4D
phrase[6]	c	0012FF4E
phrase[7]	h	0012FF4F
phrase[8]	a	0012FF50
phrase[9]	r	0012FF51
phrase[10]	a	0012FF52
phrase[11]	c	0012FF53
phrase[12]	t	0012FF54
phrase[13]	e	0012FF55
phrase[14]	r	0012FF56
phrase[15]	s	0012FF57
phrase[16]		0012FF58
phrase[17]	o	0012FF59

sPtr 0012FF48 0012FE74

```
int main()
{
    const char phrase[] =
        "print characters of a string";
    printChar( phrase );
}

void printChar( const char *sPtr )
{
    for( ; *sPtr != '\0'; sPtr++ )
        cout << *sPtr;
}
```

phrase[0]	p	0012FF48
phrase[1]	r	0012FF49
phrase[2]	i	0012FF4A
phrase[3]	n	0012FF4B
phrase[4]	t	0012FF4C
phrase[5]		0012FF4D
phrase[6]	c	0012FF4E
phrase[7]	h	0012FF4F
phrase[8]	a	0012FF50
phrase[9]	r	0012FF51
phrase[10]	a	0012FF52
phrase[11]	c	0012FF53
phrase[12]	t	0012FF54
phrase[13]	e	0012FF55
phrase[14]	r	0012FF56
phrase[15]	s	0012FF57
phrase[16]		0012FF58
phrase[17]	o	0012FF59

sPtr 0012FF49 0012FE74

```
int main()
{
    const char phrase[] =
        "print characters of a string";
    printChar( phrase );
}

void printChar( const char *sPtr )
{
    for( ; *sPtr != '\0'; sPtr++ )
        cout << *sPtr;
}
```

phrase[0]	p	0012FF48
phrase[1]	r	0012FF49
phrase[2]	i	0012FF4A
phrase[3]	n	0012FF4B
phrase[4]	t	0012FF4C
phrase[5]		0012FF4D
phrase[6]	c	0012FF4E
phrase[7]	h	0012FF4F
phrase[8]	a	0012FF50
phrase[9]	r	0012FF51
phrase[10]	a	0012FF52
phrase[11]	c	0012FF53
phrase[12]	t	0012FF54
phrase[13]	e	0012FF55
phrase[14]	r	0012FF56
phrase[15]	s	0012FF57
phrase[16]		0012FF58
phrase[17]	o	0012FF59

sPtr 0012FF4A 0012FE74

```
int main()
{
    const char phrase[] =
        "print characters of a string";
    printChar( phrase );
}

void printChar( const char *sPtr )
{
    for( ; *sPtr != '\0'; sPtr++ )
        cout << *sPtr;
}
```

phrase[0]	p	0012FF48
phrase[1]	r	0012FF49
phrase[2]	i	0012FF4A
phrase[3]	n	0012FF4B
phrase[4]	t	0012FF4C
phrase[5]		0012FF4D
phrase[6]	c	0012FF4E
phrase[7]	h	0012FF4F
phrase[8]	a	0012FF50
phrase[9]	r	0012FF51
phrase[10]	a	0012FF52
phrase[11]	c	0012FF53
phrase[12]	t	0012FF54
phrase[13]	e	0012FF55
phrase[14]	r	0012FF56
phrase[15]	s	0012FF57
phrase[16]		0012FF58
phrase[17]	o	0012FF59

sPtr 0012FF4B 0012FE74

```
int main()
{
    const char phrase[] =
        "print characters of a string";
    printChar( phrase );
}

void printChar( const char *sPtr )
{
    for( ; *sPtr != '\0'; sPtr++ )
        cout << *sPtr;
}
```

phrase[0]	p	0012FF48
phrase[1]	r	0012FF49
phrase[2]	i	0012FF4A
phrase[3]	n	0012FF4B
phrase[4]	t	0012FF4C
phrase[5]		0012FF4D
phrase[6]	c	0012FF4E
phrase[7]	h	0012FF4F
phrase[8]	a	0012FF50
phrase[9]	r	0012FF51
phrase[10]	a	0012FF52
phrase[11]	c	0012FF53
phrase[12]	t	0012FF54
phrase[13]	e	0012FF55
phrase[14]	r	0012FF56
phrase[15]	s	0012FF57
phrase[16]		0012FF58
phrase[17]	o	0012FF59

```
int main()
{
    const char phrase[] = "print characters of a string";
    printCharacters( phrase );
}
```

```
void printCharacters( const char s[] )
{
    for( unsigned int i = 0; i < strlen( s ); i++ )
        cout << s[i];
}
```

```
int main()
{
    const string phrase( "print characters of a string" );
    printCharacters( phrase );
}
```

```
void printCharacters( const string s )
{
    for( unsigned int i = 0; i < s.length(); i++ )
        cout << s[i];
}
```

```
#include <iostream>
using namespace std;

void copy1( char *, const char * );
void copy2( char *, const char * );

int main()
{
    char string1[ 10 ];
    char *string2 = "Hello";
    char string3[ 10 ];
    char string4[] = "Good Bye";

    copy1( string1, string2 );
    cout << "string1 = " << string1 << endl;

    copy2( string3, string4 );
    cout << "string3 = " << string3 << endl;
}
```

```
void copy1( char * s1, const char * s2 )
{
    for( int i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ )
        ;
}
```

```
void copy2( char *s1, const char *s2 )
{
    for( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ )
        ;
}
```



```
void copy1( char * s1, const char * s2 )
{
    for( int i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ )
        ;
}
```

```
void copy2( char *s1, const char *s2 )
{
    for( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ )
        ;
}
```

```
string1 = Hello
string3 = Good Bye
```

```
int main()
{
    char string1[ 10 ];
    char *string2 = "Hello";

    copy1( string1, string2 );
    cout << "string1 = " << string1 << endl;
}

void copy1( char * s1, const char * s2 )
{
    for( int i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ )
        ;
}
```

```
copy1( string1, string2 );
```

```
void copy1( char *s1, const char *s2 )
```

```
{  
    for( int i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ )  
        ;  
}
```

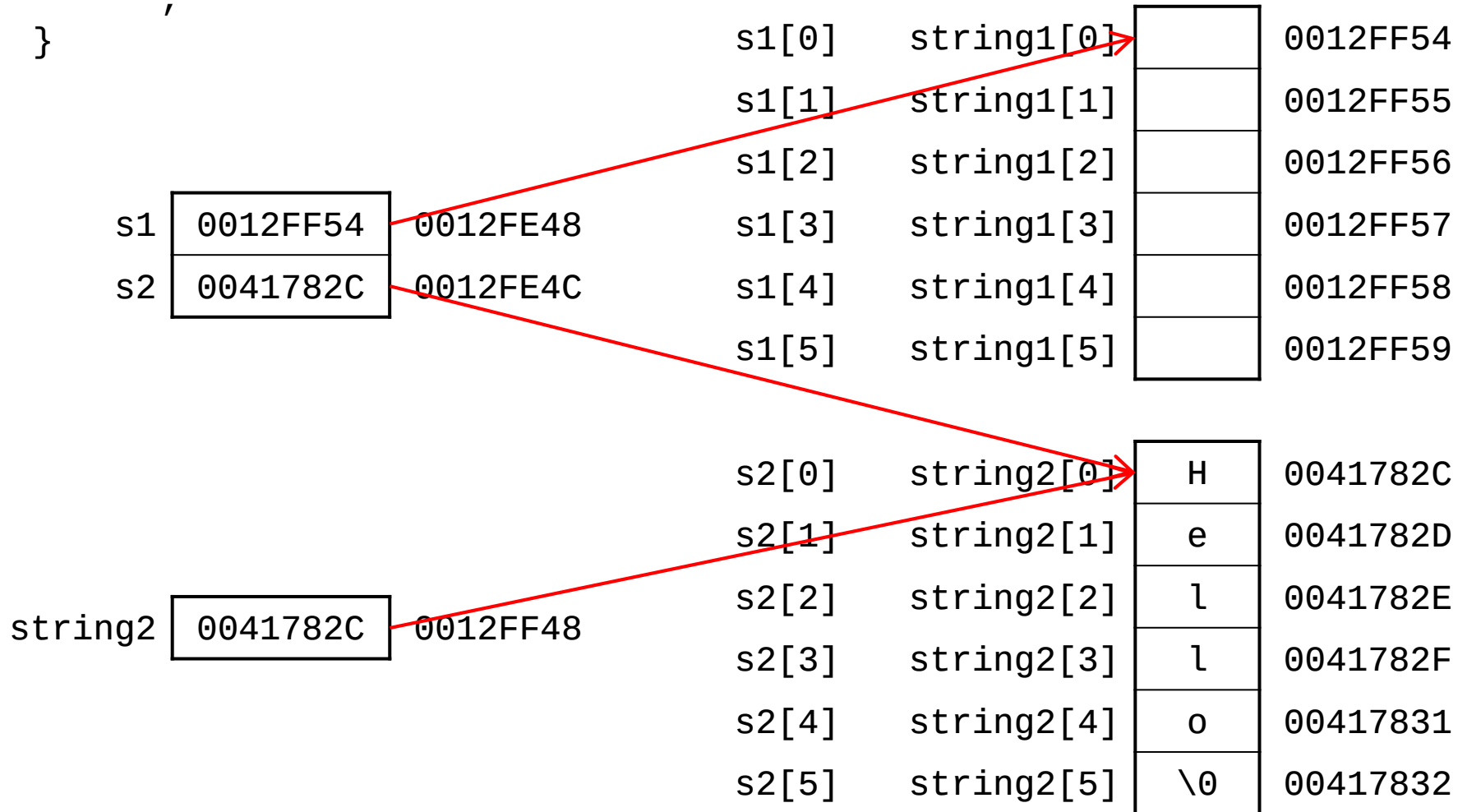
string1[0]		0012FF54
string1[1]		0012FF55
string1[2]		0012FF56
string1[3]		0012FF57
string1[4]		0012FF58
string1[5]		0012FF59

string2	0041782C	0012FF48	string2[0]	H	0041782C
			string2[1]	e	0041782D
			string2[2]	l	0041782E
			string2[3]	l	0041782F
			string2[4]	o	00417831
			string2[5]	\0	00417832

```
copy1( string1, string2 );
```

```
void copy1( char *s1, const char *s2 )
```

```
{  
    for( int i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ )  
        ;  
}
```



```
int main()
{
    char string3[ 10 ];
    char string4[] = "Good Bye";

    copy2( string3, string4 );
    cout << "string3 = " << string3 << endl;
}
```

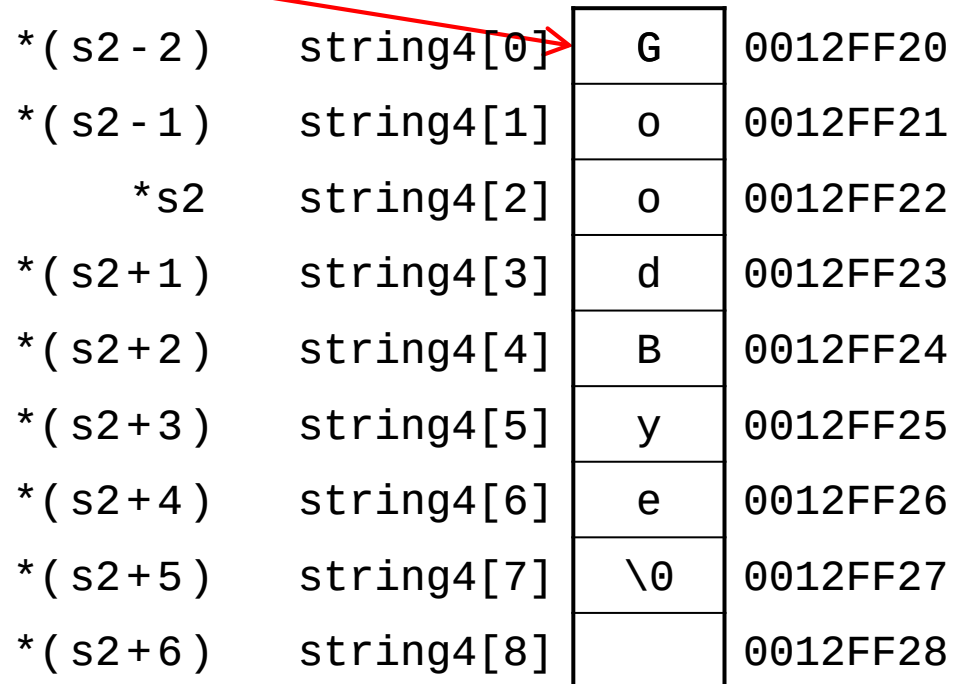
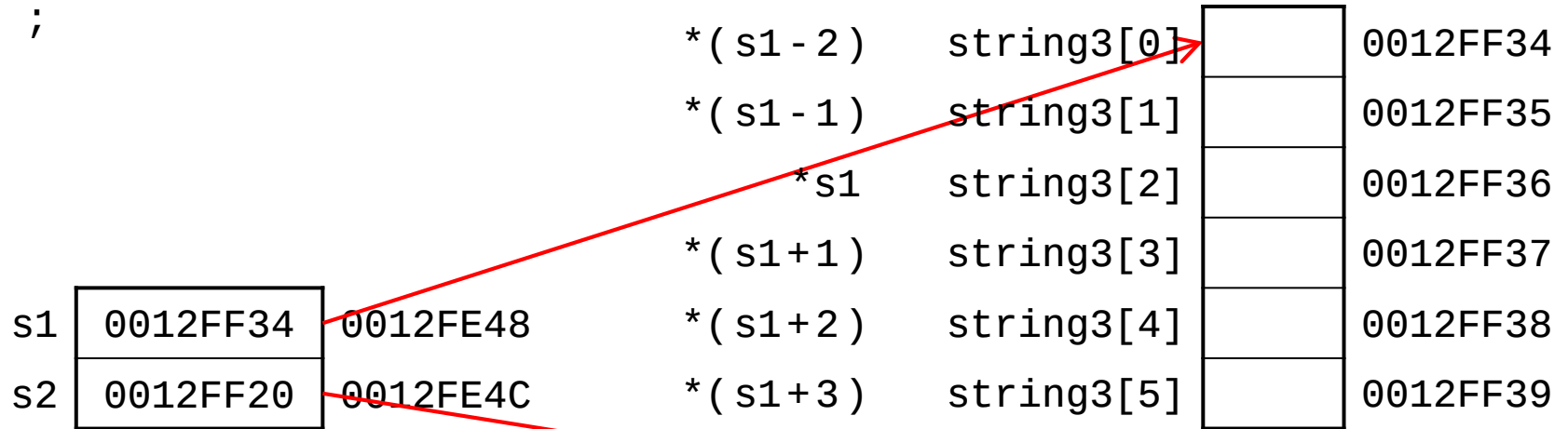
```
void copy2( char *s1, const char *s2 )
{
    for( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ )
        ;
}
```

```
for( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ )  
    ;
```

*(s1 - 2)	string3[0]		0012FF34
*(s1 - 1)	string3[1]		0012FF35
*s1	string3[2]		0012FF36
*(s1 + 1)	string3[3]		0012FF37
*(s1 + 2)	string3[4]		0012FF38
*(s1 + 3)	string3[5]		0012FF39

*(s2 - 2)	string4[0]	G	0012FF20
*(s2 - 1)	string4[1]	o	0012FF21
*s2	string4[2]	o	0012FF22
*(s2 + 1)	string4[3]	d	0012FF23
*(s2 + 2)	string4[4]	B	0012FF24
*(s2 + 3)	string4[5]	y	0012FF25
*(s2 + 4)	string4[6]	e	0012FF26
*(s2 + 5)	string4[7]	\0	0012FF27
*(s2 + 6)	string4[8]		0012FF28

```
for( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ )  
    ;
```

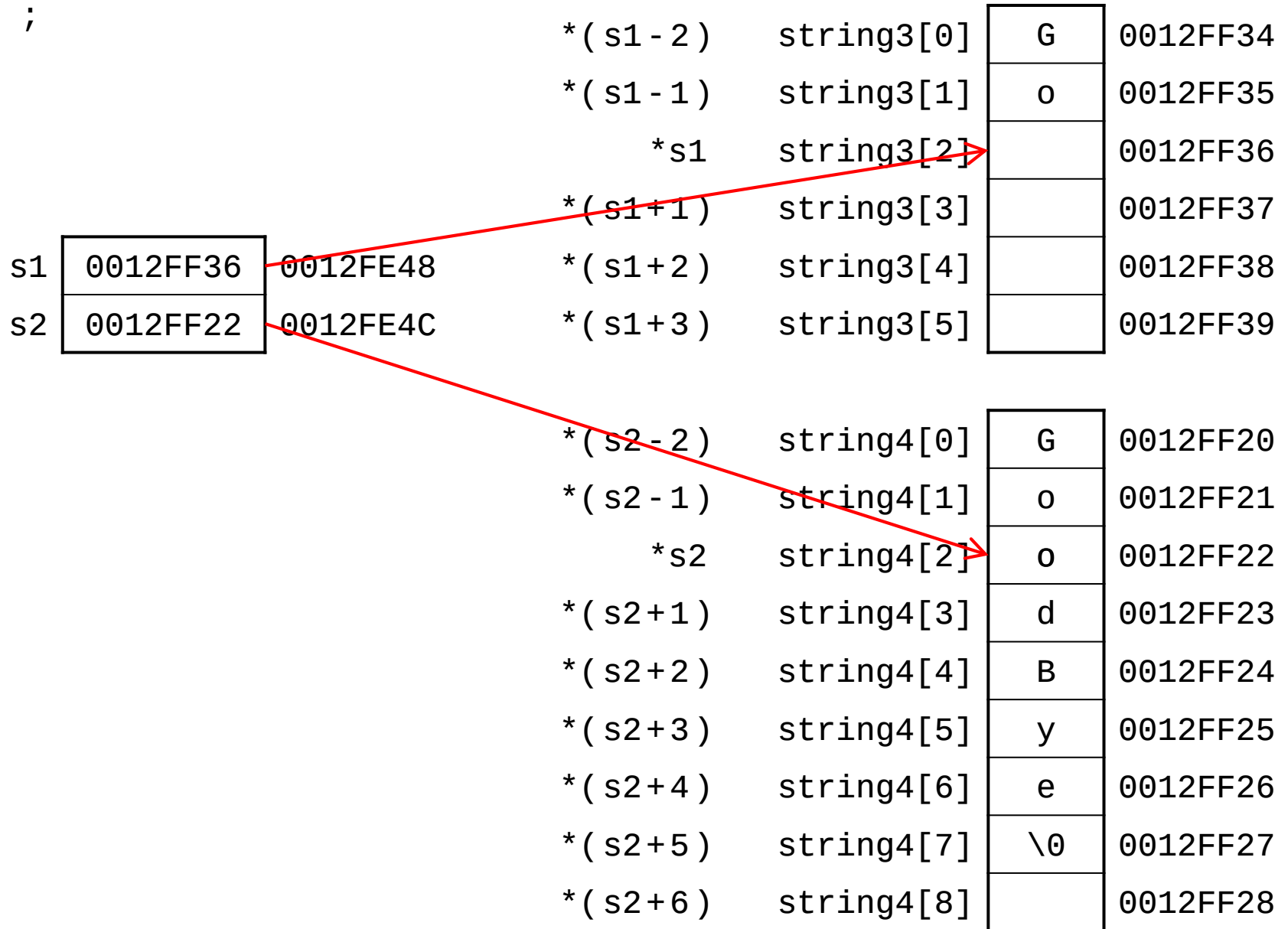


```
for( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ )  
    ;
```

		*(s1 - 2)	string3[0]	G	0012FF34
		*(s1 - 1)	string3[1]		0012FF35
		*s1	string3[2]		0012FF36
		*(s1 + 1)	string3[3]		0012FF37
s1	0012FF35	0012FE48	*(s1 + 2)	string3[4]	0012FF38
s2	0012FF21	0012FE4C	*(s1 + 3)	string3[5]	0012FF39

		*(s2 - 2)	string4[0]	G	0012FF20
		*(s2 - 1)	string4[1]	o	0012FF21
		*s2	string4[2]	o	0012FF22
		*(s2 + 1)	string4[3]	d	0012FF23
		*(s2 + 2)	string4[4]	B	0012FF24
		*(s2 + 3)	string4[5]	y	0012FF25
		*(s2 + 4)	string4[6]	e	0012FF26
		*(s2 + 5)	string4[7]	\0	0012FF27
		*(s2 + 6)	string4[8]		0012FF28


```
for( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ )
    ;
```



Pointer-Based String Manipulation Functions

```
char *strcpy( char *s1, const char *s2 );
```

Copies the string s2 into the character array s1. The value of s1 is returned.

```
char *strncpy( char *s1, const char *s2, size_t n );
```

Copies at most n characters of the string s2 into the character array s1. The value of s1 is returned.

```
char *strcat( char *s1, const char *s2 );
```

Appends the string s2 to s1. The first character of s2 overwrites the terminating null character of s1. The value of s1 is returned.

```
char *strncat( char *s1, const char *s2, size_t n );
```

Appends at most n characters of string s2 to string s1. The first character of s2 overwrites the terminating null character of s1. The value of s1 is returned.

```
int strcmp( const char *s1, const char *s2 );
```

Compares the string s1 with the string s2. The function returns a value of zero, less than zero (usually -1) or greater than zero (usually 1) if s1 is equal to, less than or greater than s2, respectively.

```
int strncmp( const char *s1, const char *s2, size_t n );
```

Compares up to `n` characters of the string `s1` with the string `s2`. The function returns zero, less than zero or greater than zero if the `n`-character portion of `s1` is equal to, less than or greater than the corresponding `n`-character portion of `s2`, respectively.

```
char *strtok( char *s1, const char *s2 );
```

A sequence of calls to `strtok` breaks string `s1` into “tokens” — logical pieces such as words in a line of text. The string is broken up based on the characters contained in string `s2`. For instance, if we were to break the string `"this:is:a:string"` into tokens based on the character `':'`, the resulting tokens would be `"this"`, `"is"`, `"a"` and `"string"`. Function `strtok` returns only one token at a time, however. The first call contains `s1` as the first argument, and subsequent calls to continue tokenizing the same string contain `NULL` as the first argument. A pointer to the current token is returned by each call. If there are no more tokens when the function is called, `NULL` is returned.

```
size_t strlen( const char *s );
```

Determines the length of string `s`. The number of characters preceding the terminating null character is returned.

Pointer-Based String Manipulation Functions

- Copying strings

- `char *strcpy( char *s1, const char *s2 )`

- Copies second argument into first argument

- First argument must be large enough to store string and terminating null character

- `char *strncpy( char *s1, const char *s2, size_t n )`

- Specifies number of characters to be copied from string into array
 - Does not necessarily copy terminating null character

```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    char x[] = "Happy Birthday to You";
    char y[ 25 ];
    char z[ 15 ];

    strcpy( y, x );

    cout << "The string in array x is: " << x
         << "\nThe string in array y is: " << y << '\n';

    strncpy( z, x, 14 ); // copy first 14 characters of x into z
    z[ 14 ] = '\0'; // append '\0' to z's contents

    cout << "The string in array z is: " << z << endl;
}
```

```
strcpy( y, x );
```

y[0]		0012FF4C	x[0]	H	0012FF68
y[1]		0012FF4D	x[1]	a	0012FF69
y[2]		0012FF4E	x[2]	p	0012FF6A
y[3]		0012FF4F	x[3]	p	0012FF6B
y[4]		0012FF50	x[4]	y	0012FF6C
y[5]		0012FF51	x[5]		0012FF6D
y[6]		0012FF52	x[6]	B	0012FF6E
y[7]		0012FF53	x[7]	i	0012FF6F
y[8]		0012FF54	x[8]	r	0012FF70
y[9]		0012FF55	x[9]	t	0012FF71
y[10]		0012FF56	x[10]	h	0012FF72
y[11]		0012FF57	x[11]	d	0012FF73
y[12]		0012FF58	x[12]	a	0012FF74
y[13]		0012FF59	x[13]	y	0012FF75
y[14]		0012FF5A	x[14]		0012FF76
y[15]		0012FF5B	x[15]	t	0012FF77
y[16]		0012FF5C	x[16]	o	0012FF78
y[17]		0012FF5D	x[17]		0012FF79
y[18]		0012FF5E	x[18]	Y	0012FF7A
y[19]		0012FF5F	x[19]	o	0012FF7B
y[20]		0012FF60	x[20]	u	0012FF7C
y[21]		0012FF61	x[21]	/0	0012FF7D

```
int main()
{
    char x[] = "Happy Birthday to You"; // string length 21
    char y[ 25 ];
    char z[ 15 ];

    strcpy( y, x ); // copy contents of x into y

    cout << "The string in array x is: " << x
          << "\nThe string in array y is: " << y << '\n';

    // copy first 14 characters of x into z
    strncpy( z, x, 14 ); // does not copy null character
    z[ 14 ] = '\0'; // append '\0' to z's contents

    cout << "The string in array z is: " << z << endl;
}
```



```
int main()
{
    char x[] = "Happy Birthday to You"; // string length 21
    char y[ 25 ];
    char z[ 15 ];

    strcpy_s( y, 25, x ) // copy contents of x into y

    cout << "The string in array x is: " << x
         << "\nThe string in array y is: " << y << '\n';

    // copy first 14 characters of x into z
    strncpy_s( z, 15, x, 14 ); // places '\0' after last character

    cout << "The string in array z is: " << z << endl;
}
```

```
int main()
{
    string x( "Happy Birthday to You" ); // string length 21
    string y;
    string z;

    y = x; // copy contents of x into y

    cout << "The string in array x is: " << x
          << "\nThe string in array y is: " << y << '\n';

    // copy first 14 characters of x into z
    z = x.substr( 0, 14 );

    cout << "The string in array z is: " << z << endl;
}
```

The string in array x is: Happy Birthday to You

The string in array y is: Happy Birthday to You

The string in array z is: Happy Birthday

Pointer-Based String Manipulation Functions

- Concatenating strings

- `char *strcat( char *s1, const char *s2 )`

- Appends second argument to first argument
 - First character of second argument replaces null character terminating first argument
 - Ensure first argument large enough to store concatenated result and null character

- `char *strncat( char *s1, const char *s2, size_t n )`

- Appends specified number of characters from second argument to first argument
 - Appends terminating null character to result

```

int main()
{
    char s1[ 20 ] = "Happy ";
    char s2[] = "New Year ";
    char s3[ 40 ] = "";

    cout << "s1 = " << s1 << "\ns2 = " << s2;

    strcat( s1, s2 );

    cout << "\n\nAfter strcat(s1, s2):\ns1 = " << s1 << "\ns2 = " << s2;

    // concatenate first 6 characters of s1 to s3
    strncat( s3, s1, 6 ); // places '\0' after last character

    cout << "\n\nAfter strncat(s3, s1, 6):\ns1 = " << s1
        << "\ns3 = " << s3;

    strcat( s3, s1 );
    cout << "\n\nAfter strcat(s3, s1):\ns1 = " << s1
        << "\ns3 = " << s3 << endl;
}

```

```
s1 = Happy  
s2 = New Year
```

```
After strcat(s1, s2):  
s1 = Happy New Year  
s2 = New Year
```

```
After strncat(s3, s1, 6):  
s1 = Happy New Year  
s3 = Happy
```

```
After strcat(s3, s1):  
s1 = Happy New Year  
s3 = Happy Happy New Year
```

```
strcat( s1, s2 );
```

		0012FF58	s1[0]	H	0012FF6C
		0012FF59	s1[1]	a	0012FF6D
		0012FF5A	s1[2]	p	0012FF6E
		0012FF5B	s1[3]	p	0012FF6F
		0012FF5C	s1[4]	y	0012FF70
		0012FF5D	s1[5]		0012FF71
		0012FF5E	s1[6]	/0	0012FF72
		0012FF5F	s1[7]		0012FF73
s2[0]	N	0012FF60	s1[8]		0012FF74
s2[1]	e	0012FF61	s1[9]		0012FF75
s2[2]	w	0012FF62	s1[10]		0012FF76
s2[3]		0012FF63	s1[11]		0012FF77
s2[4]	Y	0012FF64	s1[12]		0012FF78
s2[5]	e	0012FF65	s1[13]		0012FF79
s2[6]	a	0012FF66	s1[14]		0012FF7A
s2[7]	r	0012FF67	s1[15]		0012FF7B
s2[8]		0012FF68	s1[16]		0012FF7C
s2[9]	/0	0012FF69	s1[17]		0012FF7D
		0012FF6A	s1[18]		0012FF7E
		0012FF6B	s1[19]		0012FF7F

```
int main()
{
    char s1[ 20 ] = "Happy ";
    char s2[] = "New Year ";
    char s3[ 40 ] = "";

    cout << "s1 = " << s1 << "\ns2 = " << s2;

    strcat( s1, s2 ); // concatenate s2 to s1 (length 15)

    cout << s1 << "\ns2 = " << s2;

    strncat( s3, s1, 6 ); // places '\0' after last character

    cout << s1 << "\ns3 = " << s3;

    strcat( s3, s1 ); // concatenate s1 to s3
    cout << s1 << "\ns3 = " << s3 << endl;
}
```



```
int main()
{
    char s1[ 20 ] = "Happy ";
    char s2[] = "New Year ";
    char s3[ 40 ] = "";

    cout << "s1 = " << s1 << "\ns2 = " << s2;

    strcat_s( s1, 20, s2 ); // concatenate s2 to s1 (length 15)

    cout << s1 << "\ns2 = " << s2;

    strncat_s( s3, 40, s1, 6 ); // places '\0' after last character

    cout << s1 << "\ns3 = " << s3;

    strcat_s( s3, 40, s1 ); // concatenate s1 to s3
    cout << s1 << "\ns3 = " << s3 << endl;
}
```

```
int main()
{
    string s1( "Happy " );
    string s2( "New Year " );
    string s3;

    cout << "s1 = " << s1 << "\ns2 = " << s2;

    s1 += s2; // concatenate s2 to s1 (length 15)

    cout << s1 << "\ns2 = " << s2;

    s3 += s1.substr( 0, 6 );

    cout << s1 << "\ns3 = " << s3;

    s3 += s1; // concatenate s1 to s3
    cout << s1 << "\ns3 = " << s3 << endl;
}
```

Pointer-Based String Manipulation Functions

- Comparing strings
 - Characters represented as numeric codes
 - Strings compared using numeric codes
 - ASCII character sets
 - “American Standard Code for Information Interchange”

Pointer-Based String Manipulation Functions

- Comparing strings

- `int strcmp( const char *s1, const char *s2 )`

- Compares character by character
 - Returns
 - Zero if strings equal
 - Negative value if first string less than second string
 - Positive value if first string greater than second string

- `int strncmp(const char *s1,
const char *s2, size_t n)`

- Compares up to specified number of characters
 - Stops comparing if reaches null character in one of arguments

```

#include <iostream>
#include <iomanip>
#include <cstring>
using namespace std;

int main()
{
    char *s1 = "Happy New Year";
    char *s2 = "Happy New Year";
    char *s3 = "Happy Holidays";

    cout << "s1 = " << s1 << "\ns2 = " << s2 << "\ns3 = " << s3
         << "\n\nstrcmp(s1, s2) = " << setw( 2 ) << strcmp( s1, s2 )
         << "\nstrcmp(s1, s3) = " << setw( 2 ) << strcmp( s1, s3 )
         << "\nstrcmp(s3, s1) = " << setw( 2 ) << strcmp( s3, s1 );

    cout << "\n\nstrncmp(s1, s3, 6) = " << setw( 2 )
         << strncmp( s1, s3, 6 ) << "\nstrncmp(s1, s3, 7) = " << setw( 2 )
         << strncmp( s1, s3, 7 ) << "\nstrncmp(s3, s1, 7) = " << setw( 2 )
         << strncmp( s3, s1, 7 ) << endl;
}

```

```
s1 = Happy New Year  
s2 = Happy New Year  
s3 = Happy Holidays
```

```
strcmp(s1, s2) = 0  
strcmp(s1, s3) = 1  
strcmp(s3, s1) = -1
```

```
strncmp(s1, s3, 6) = 0  
strncmp(s1, s3, 7) = 1  
strncmp(s3, s1, 7) = -1
```

Pointer-Based String Manipulation Functions

- Determining string lengths
 - `size_t strlen( const char *s )`
 - Returns number of characters in string
 - Terminating null character not included in length

```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    char *string1 = "abcdefghijklmnopqrstuvwxyz";
    char *string2 = "four";
    char *string3 = "Boston";

    cout << "The length of \"" << string1 << "\" is " << strlen( string1 )
        << "\nThe length of \"" << string2 << "\" is " << strlen( string2 )
        << "\nThe length of \"" << string3 << "\" is " << strlen( string3 )
        << endl;
}
```


The length of "abcdefghijklmnopqrstuvwxyz" is 26

The length of "four" is 4

The length of "Boston" is 6

```
int main()
{
    char *string1 = "abcdefghijklmnopqrstuvwxyz";
    char *string2 = "four";
    char *string3 = "Boston";

    cout << "The length of \"" << string1 << "\" " << strlen( string1 )
        << "\nThe length of \"" << string2 << "\" " << strlen( string2 )
        << "\nThe length of \"" << string3 << "\" " << strlen( string3 );
}
```

```
int main()
{
    string string1( "abcdefghijklmnopqrstuvwxyz" );
    string string2( "four" );
    string string3( "Boston" );

    cout << "The length of \"" << string1 << "\" " << string1.length()
        << "\nThe length of \"" << string2 << "\" " << string2.length()
        << "\nThe length of \"" << string3 << "\" " << string3.length();
}
```

Pointer-Based String Manipulation Functions

- Tokenizing
 - Breaking strings into tokens, separated by delimiting characters
 - Tokens usually logical units, such as words (separated by spaces)
 - "This is my string" has 4 word tokens (separated by spaces)
 - `char *strtok( char *s1, const char *s2 )`
 - Multiple calls required
 - First call contains two arguments, string to be tokenized and string containing delimiting characters
 - Finds next delimiting character and replaces with null character
 - Subsequent calls continue tokenizing
 - Call with first argument NULL

```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    {
        char string[] = "Programming is learned by writing programs";

        cout << string << endl << endl;

        char *token = strtok( string, " " ); // get the first token

        while( token != nullptr )
        {
            cout << token << endl;
            token = strtok( nullptr, " " ); // get next token
        }
        cout << endl;

        cout << string << endl << endl;
    }
}
```

```
#include <iostream>
#include <cstring>
using namespace std;

int main()
{
    {
        char string[] = "Programming is learned by writing programs";

        cout << string << endl << endl;

        char *nextToken = nullptr;
        char *token = strtok_s( string, " ", &nextToken );

        while( token != nullptr )
        {
            cout << token << endl;
            token = strtok_s( nullptr, " ", &nextToken );
        }
        cout << endl;
        cout << string << endl << endl;
    }
}
```

```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```



Output

ptr
□

□
nextToken

H
o
w
a
r
e
y
o
u
\0

```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```



Output

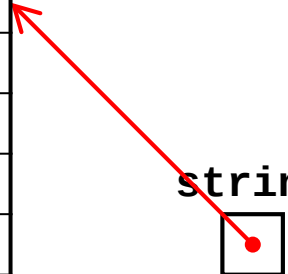
ptr
□

□
nextToken

H
o
w
a
r
e
y
o
u
\0

string
□

□
str



```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

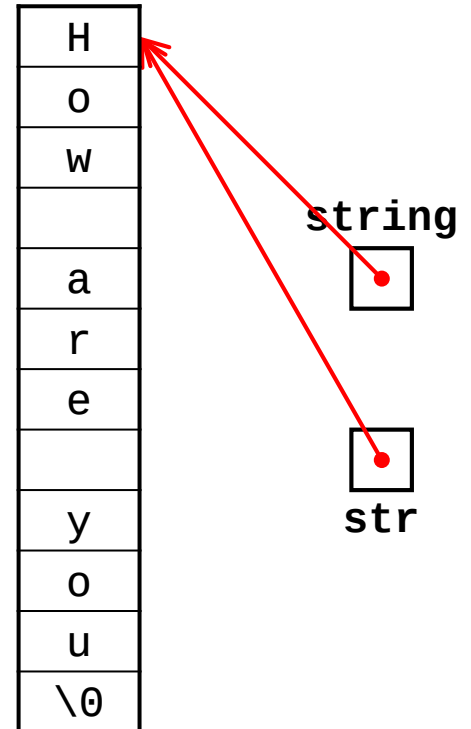
```



Output

ptr
□

□
nextToken




```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

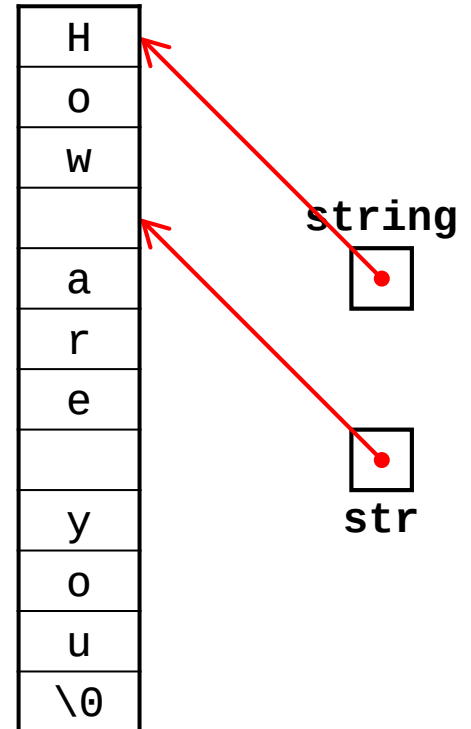
```



Output

ptr
□

□
nextToken



```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

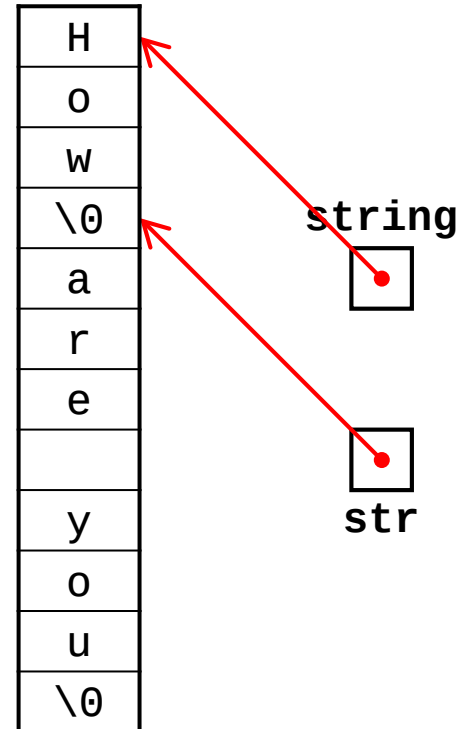
```



Output

ptr
□

□
nextToken



```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

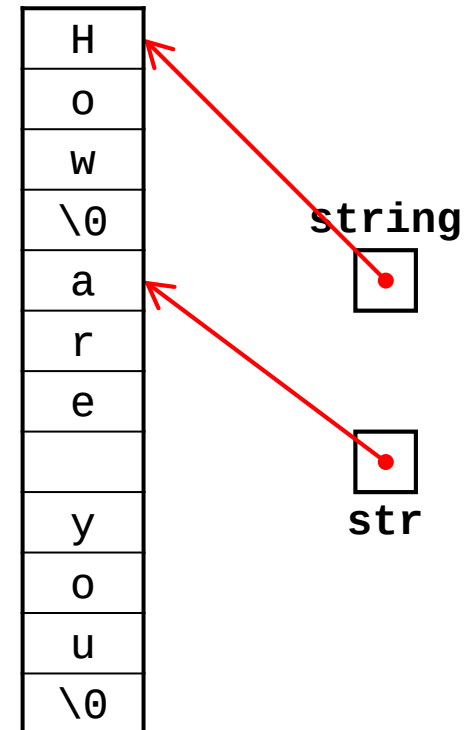
```



Output

ptr
□

□
nextToken



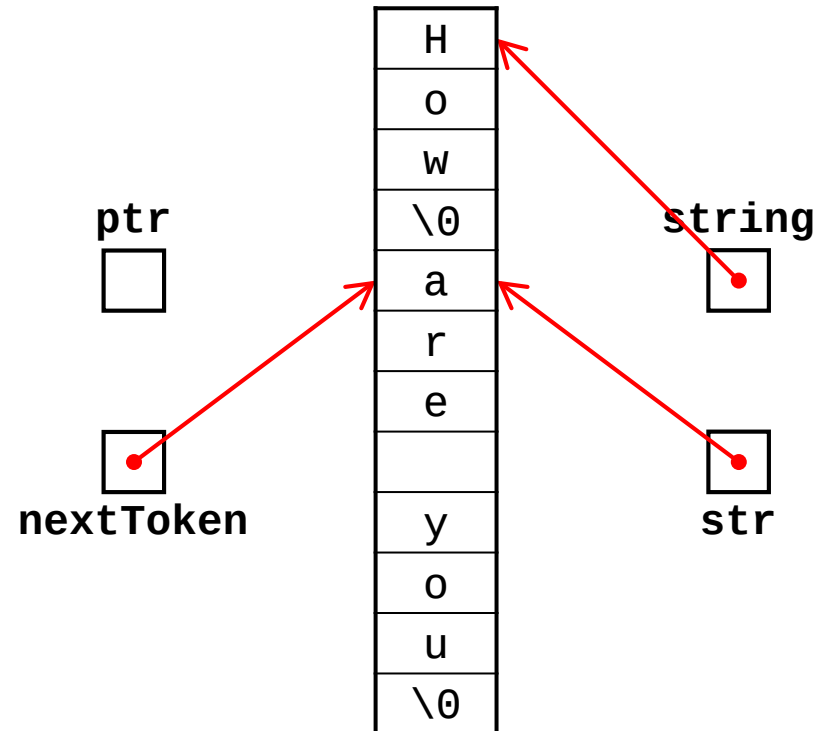
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```



Output



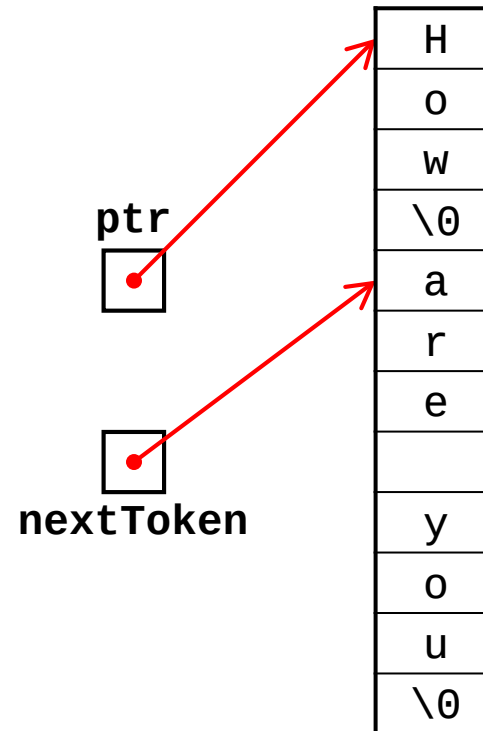
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```



Output



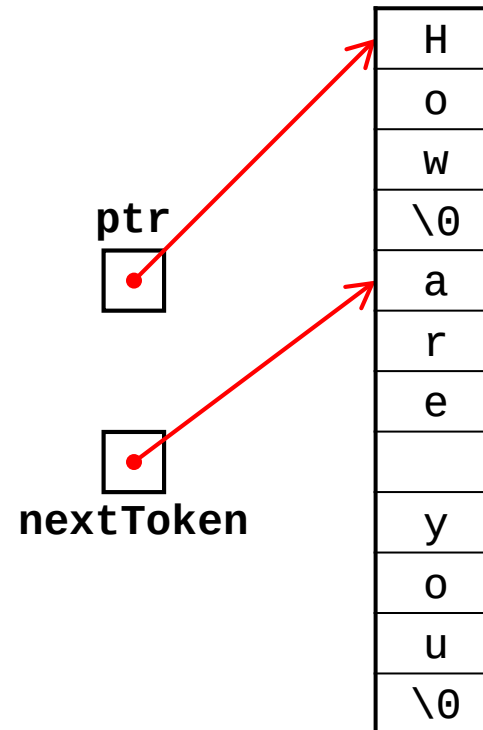
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How

Output



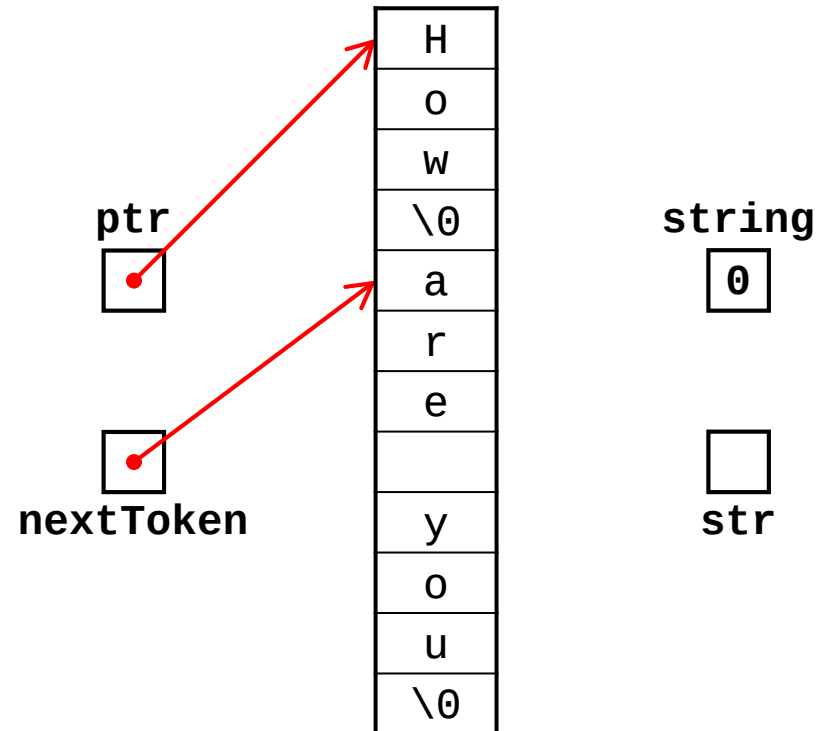
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How

Output



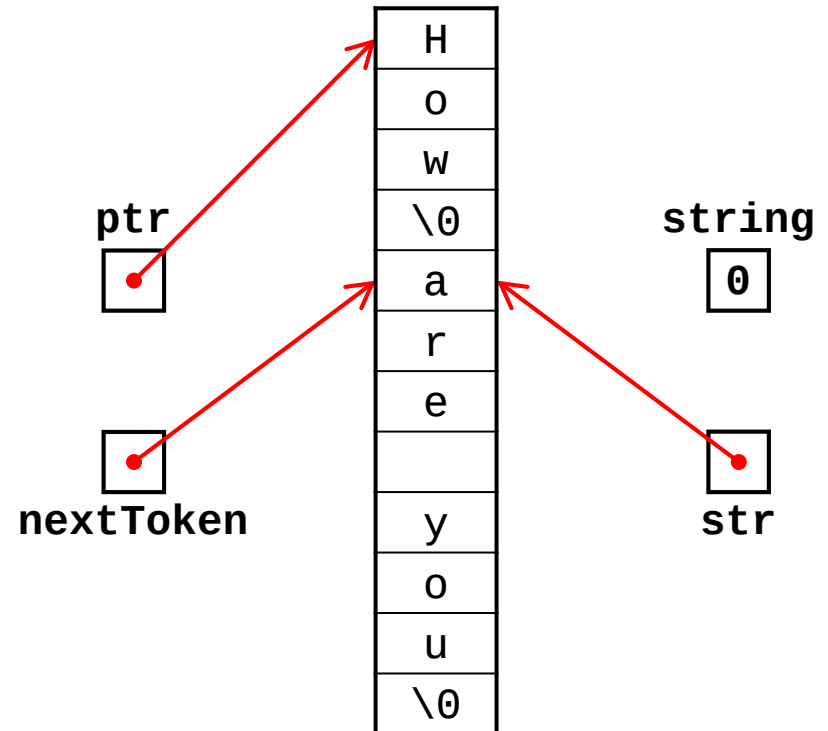
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How

Output



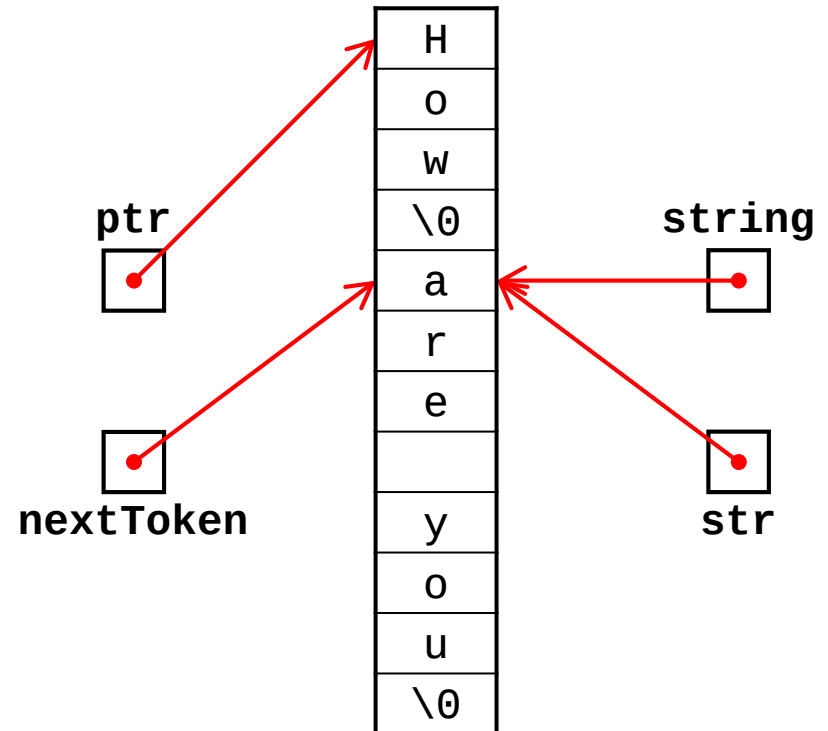

```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How

Output



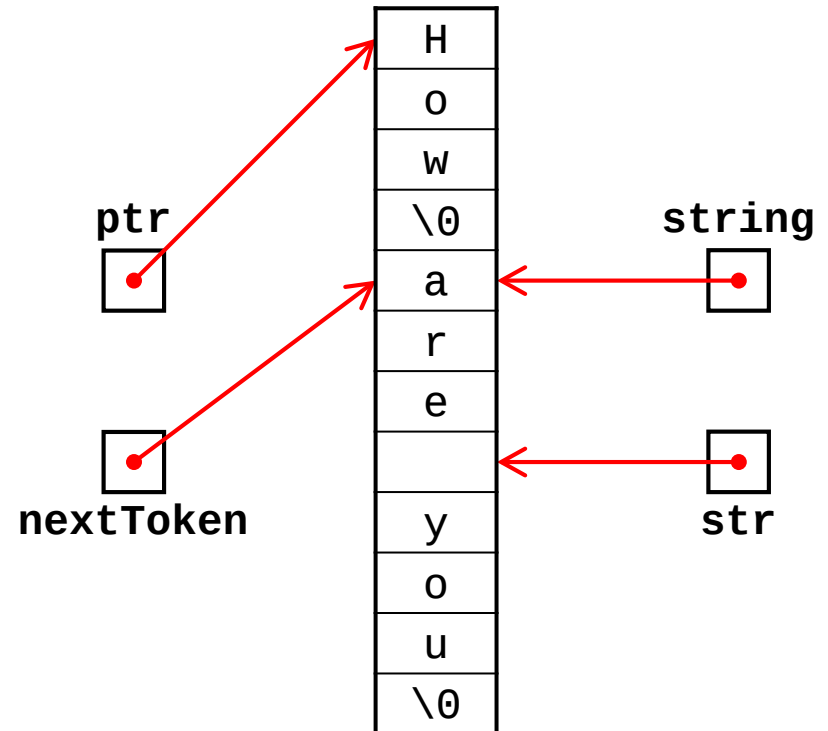
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How

Output



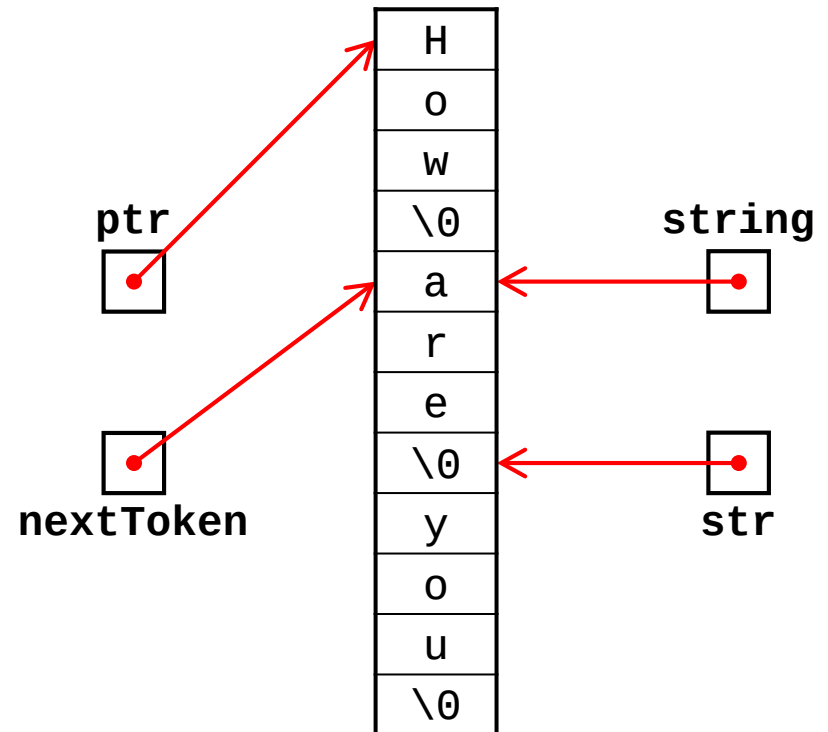
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How

Output



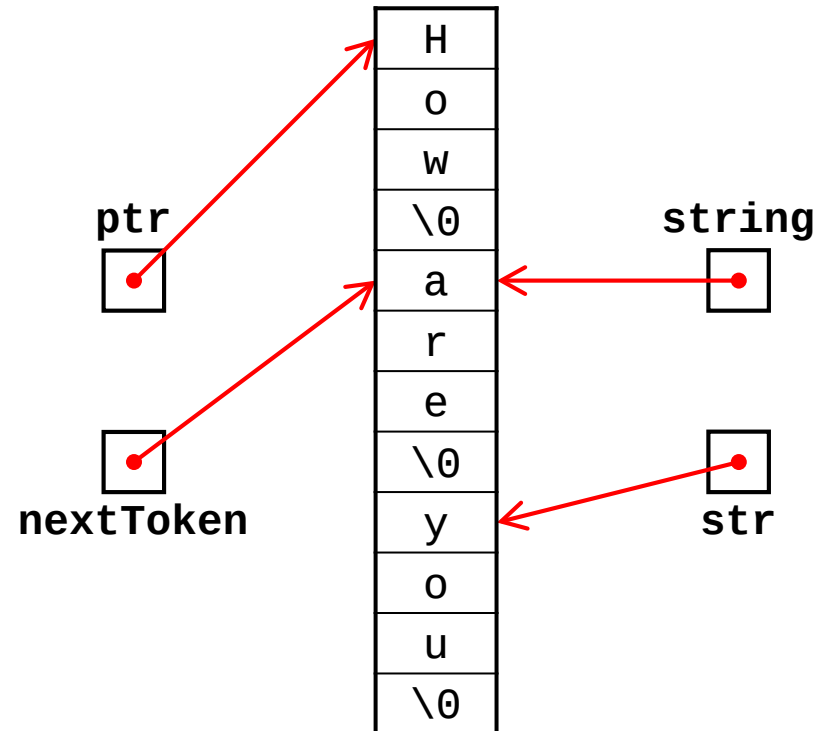
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How

Output



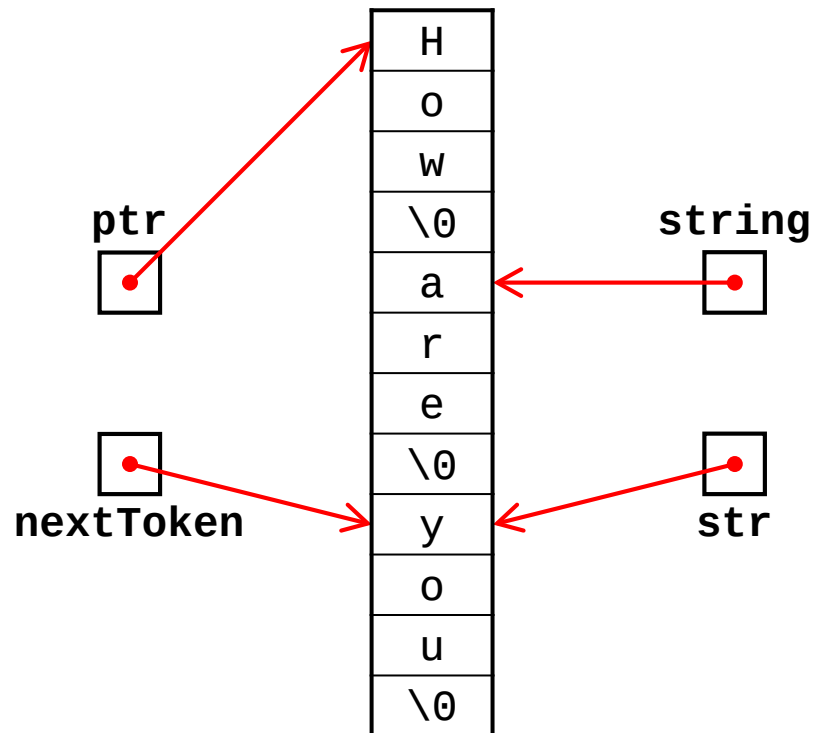
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How

Output



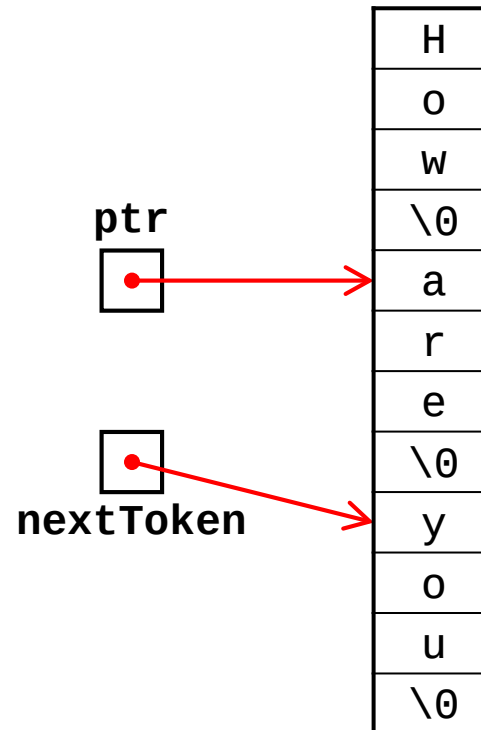
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How

Output



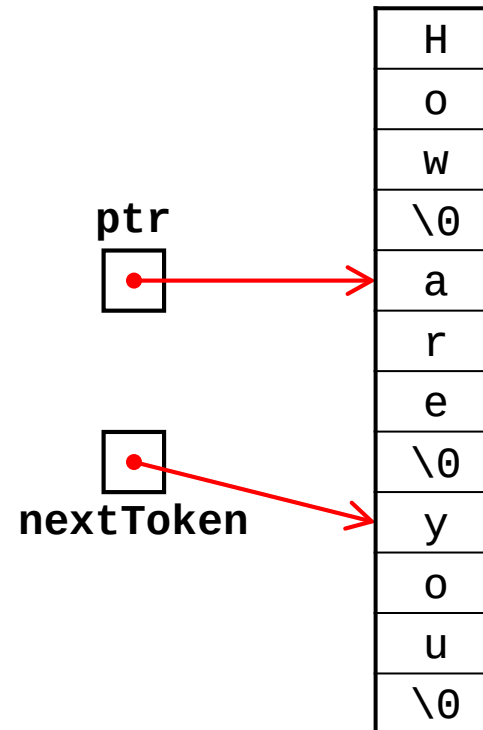
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are

Output



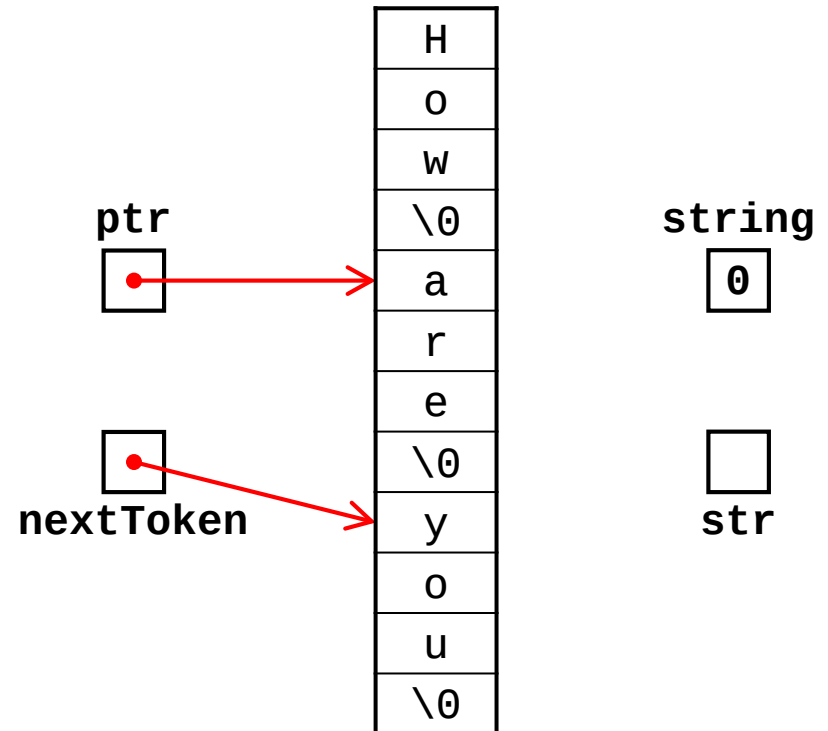
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are

Output



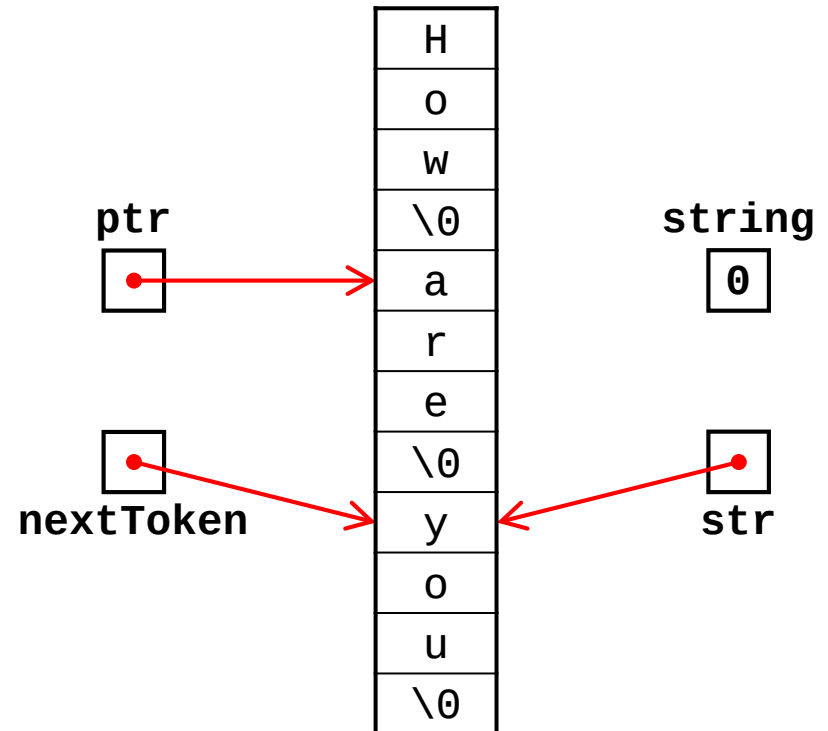

```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are

Output



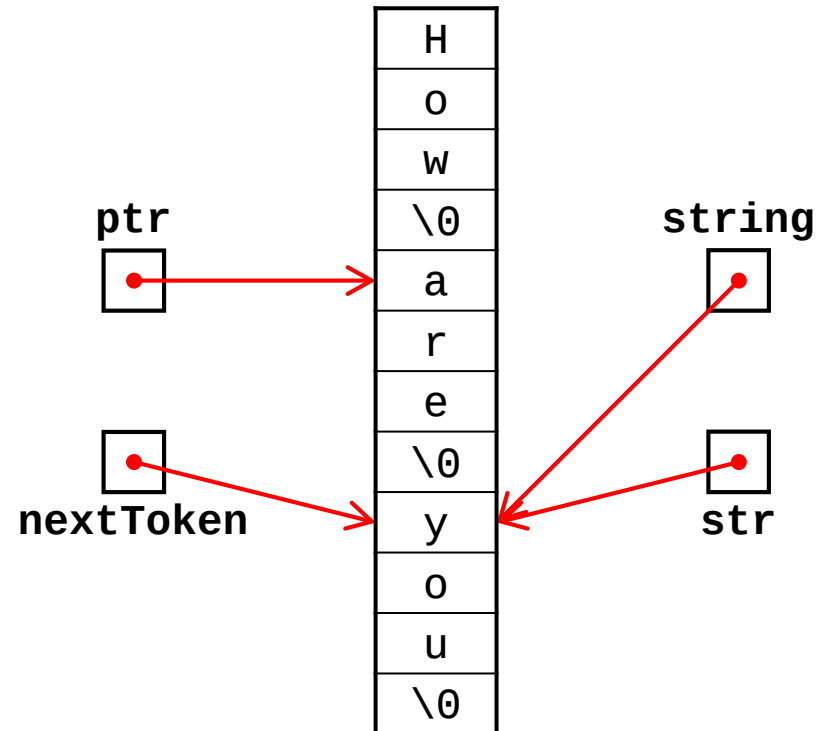
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are

Output



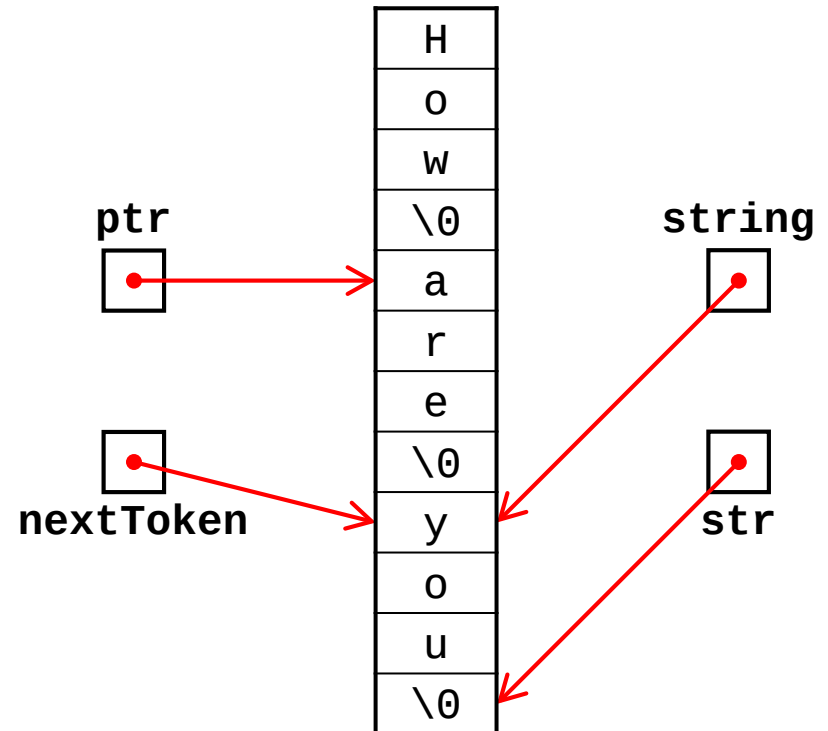
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are

Output



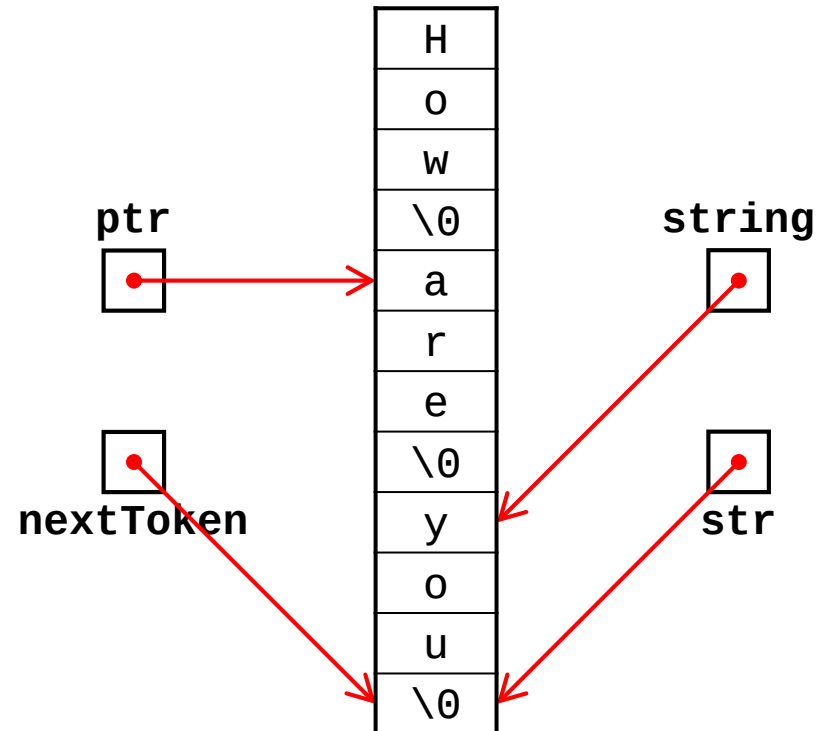
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are

Output



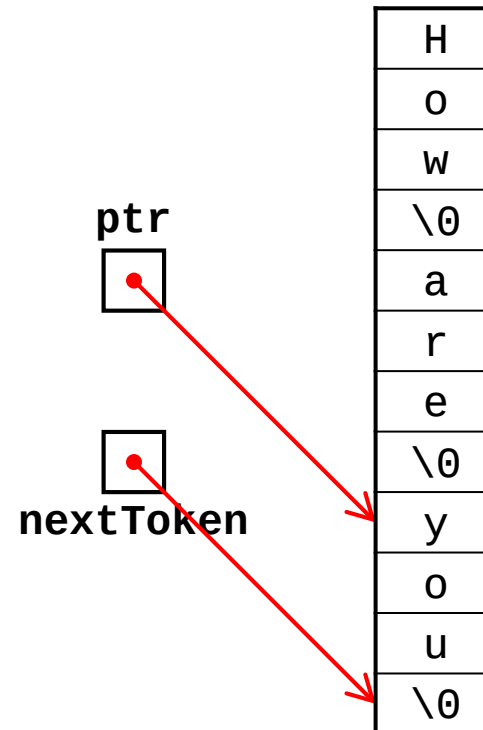
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are

Output



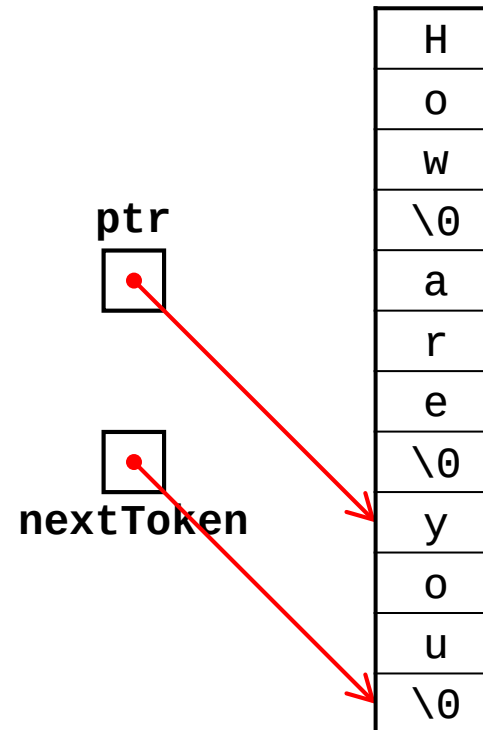
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are
you

Output



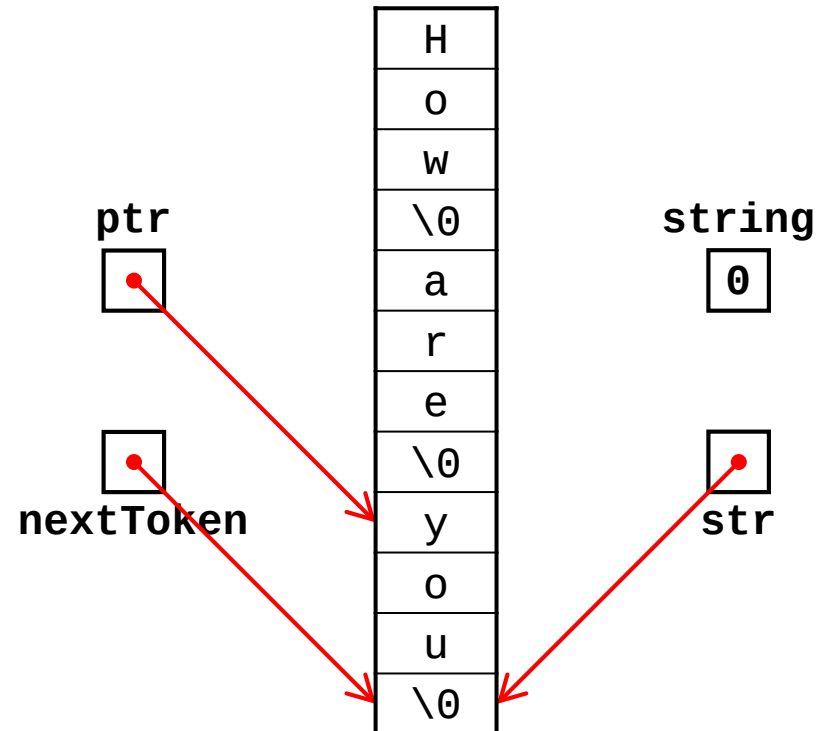
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are
you

Output



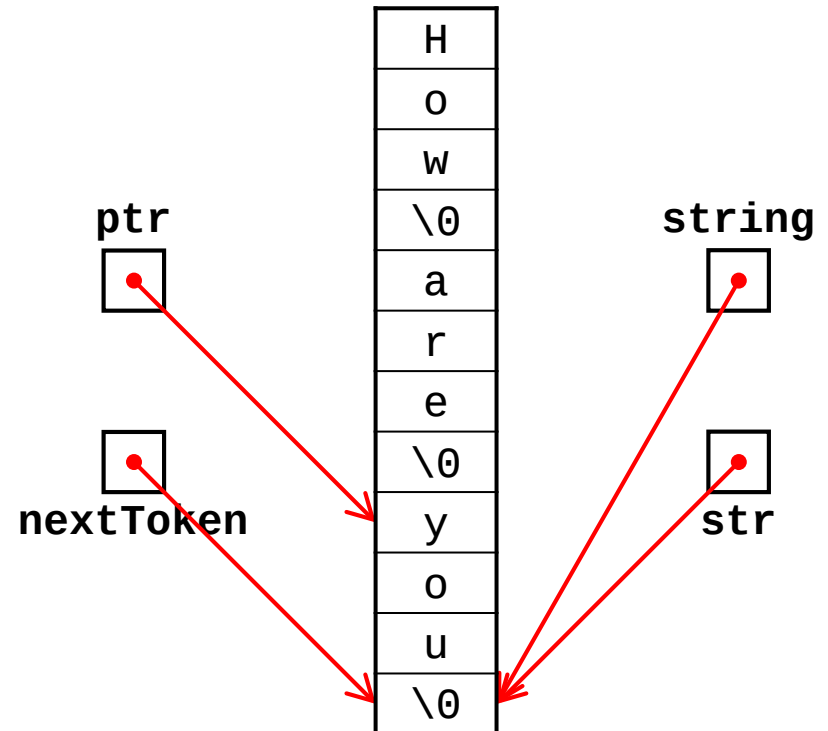
```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are
you

Output



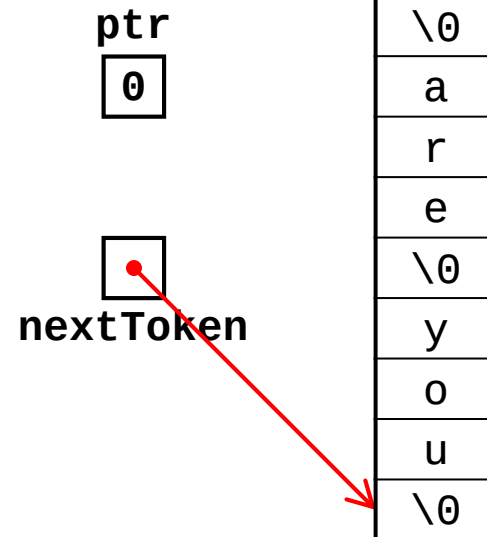

```

int main()
{
    char string[] = "How are you";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, " ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, " ", &nextToken );
    }
}

```

How
are
you

Output




```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output

ptr



nextToken



"
H
e
l
l
o
,
w
o
r
l
d
!
"
\0

```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output

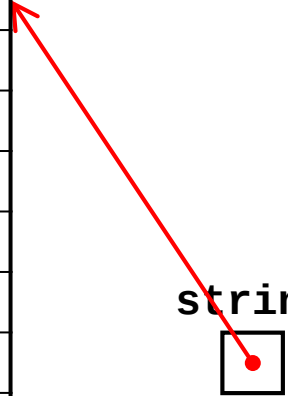
ptr
□

□
nextToken

"
H
e
l
l
o
,
w
o
r
l
d
!
"
\0

string
□

□
str



```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output

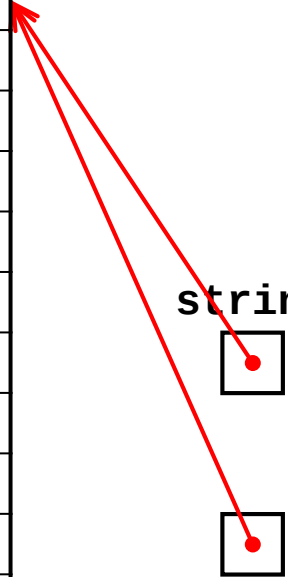
ptr
□

□
nextToken

"
H
e
l
l
o
,
w
o
r
l
d
!
"
\0

string
□

□
str



```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output

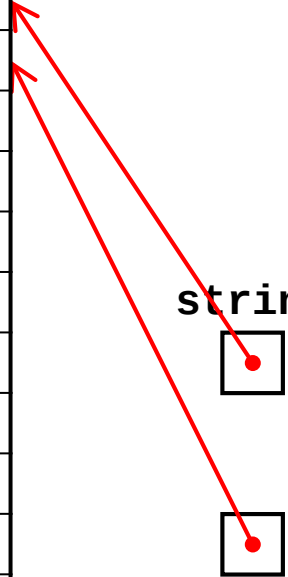
ptr
□

□
nextToken

"
H
e
l
l
o
,
w
o
r
l
d
!
"
\0

string
□

□
str



```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output

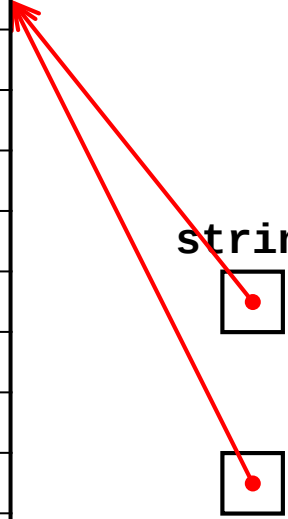
ptr
□

□
nextToken

"
H
e
l
l
o
,
w
o
r
l
d
!
"
\0

string
□

□
str



```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output

ptr
□

□
nextToken

"
H
e
l
l
o
,
w
o
r
l
d
!
"
\0

string
□

□
str




```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output

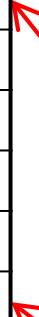
ptr
□

□
nextToken

"
H
e
l
l
o
\0
w
o
r
l
d
!
"
\0

string
□

□
str



```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output

ptr
□

□
nextToken

"
H
e
l
l
o
\0
w
o
r
l
d
!
"
\0

string
□

□
str



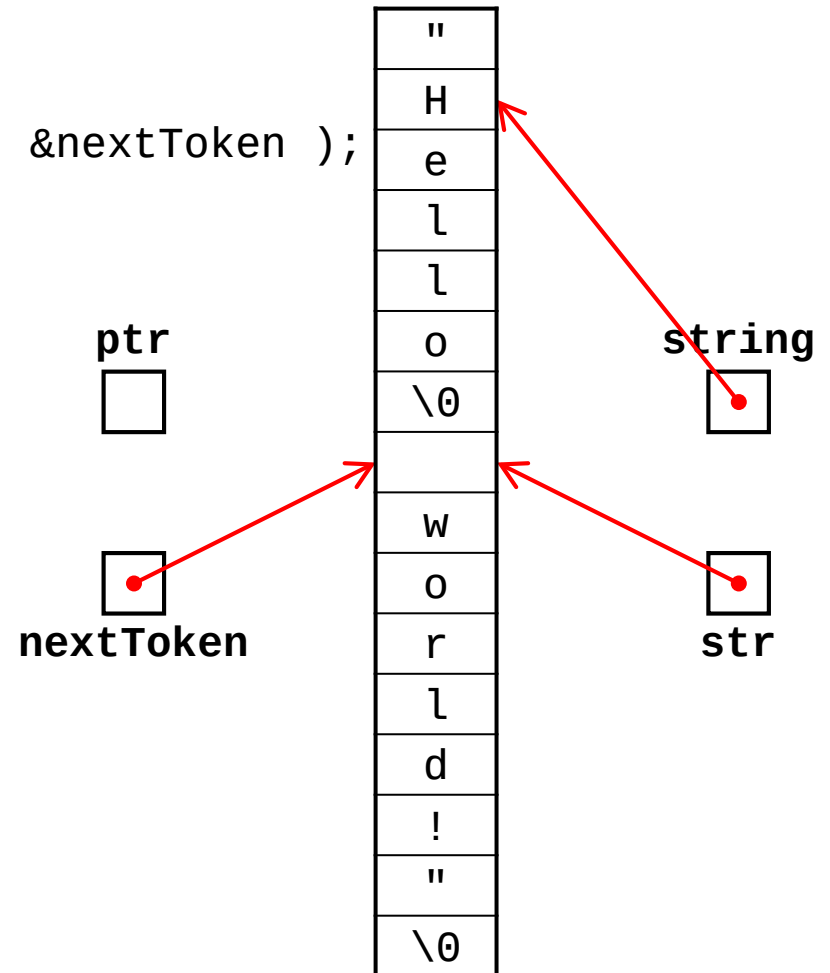
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output



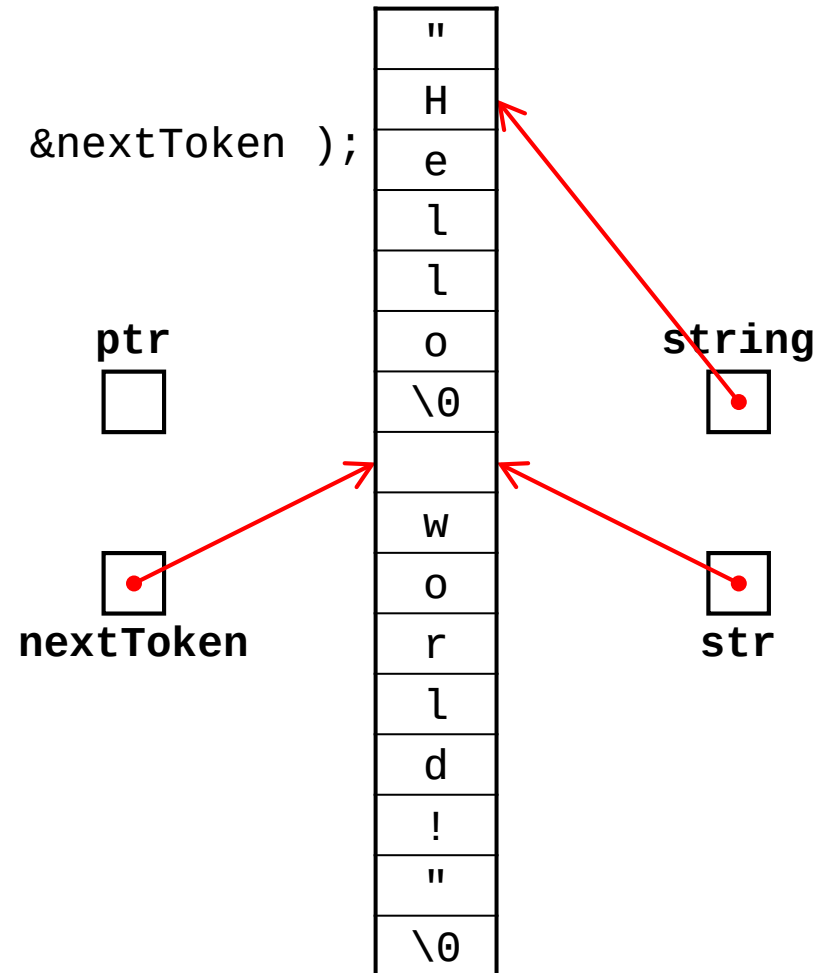
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output



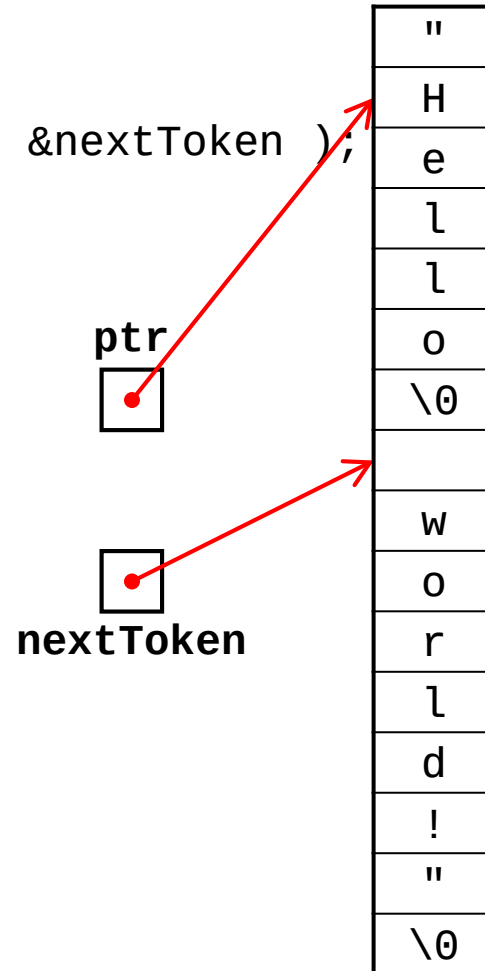
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```



Output



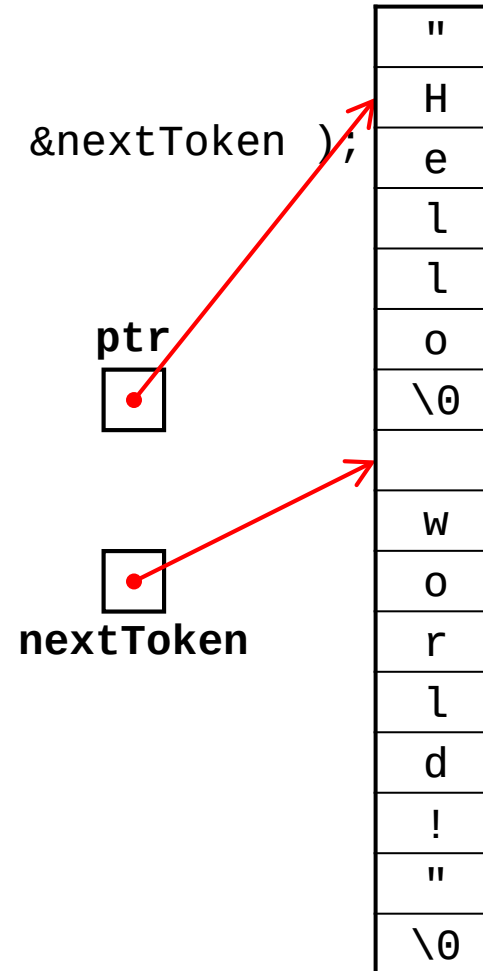
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



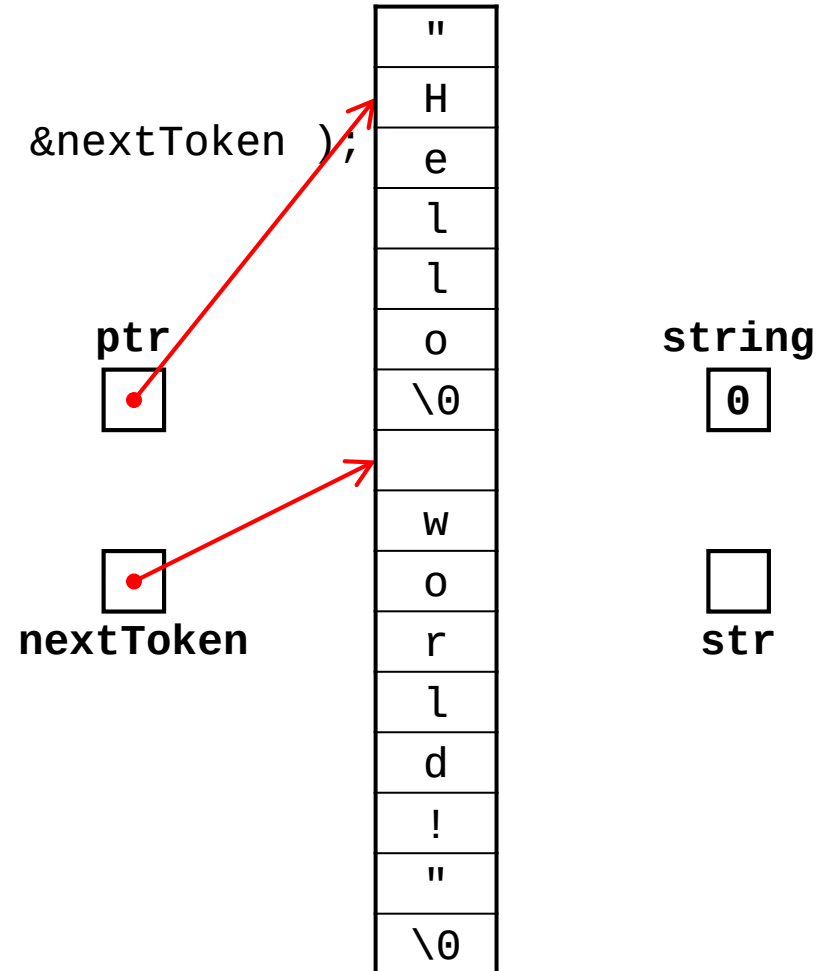
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



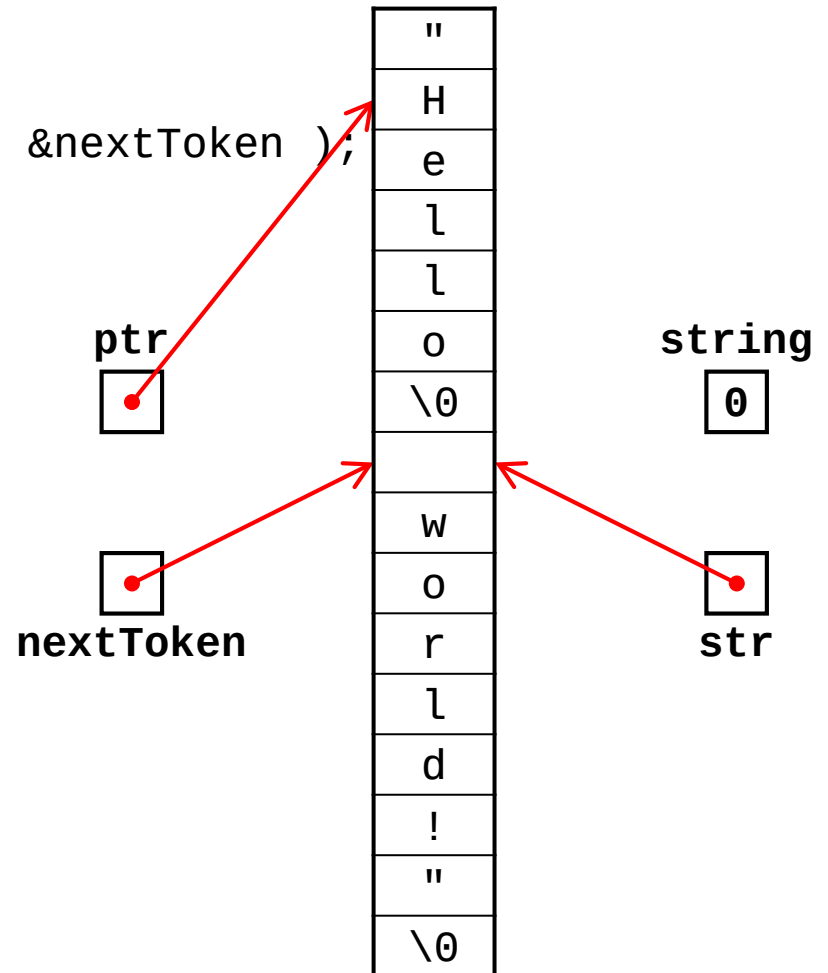
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



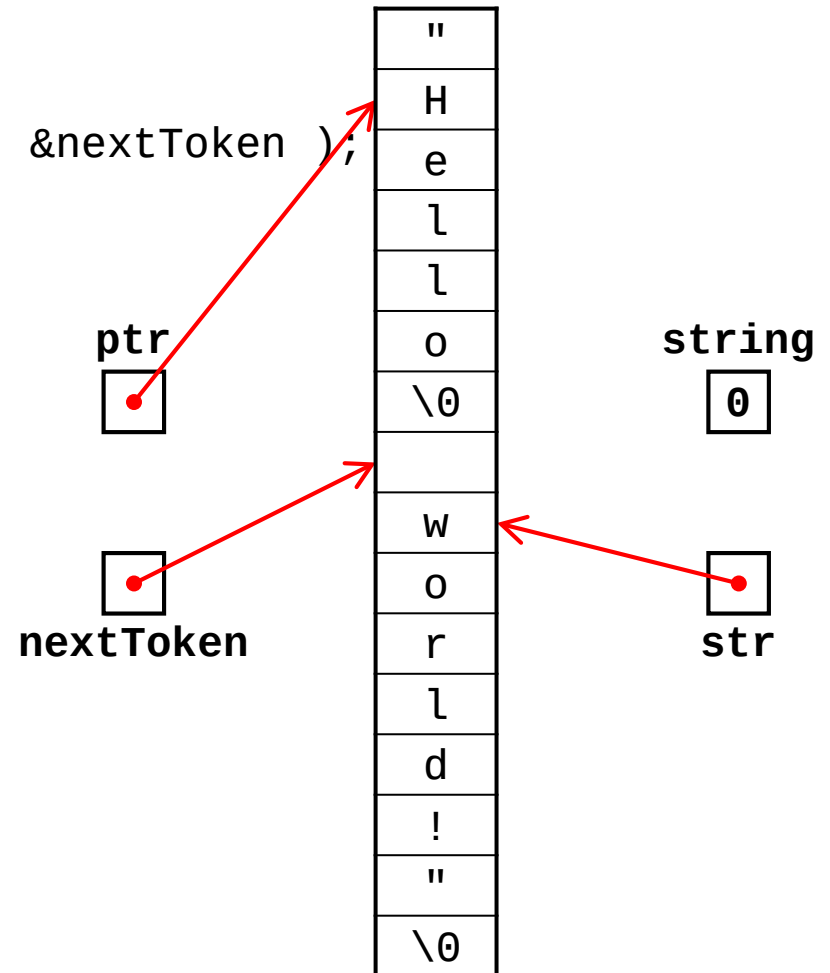

```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



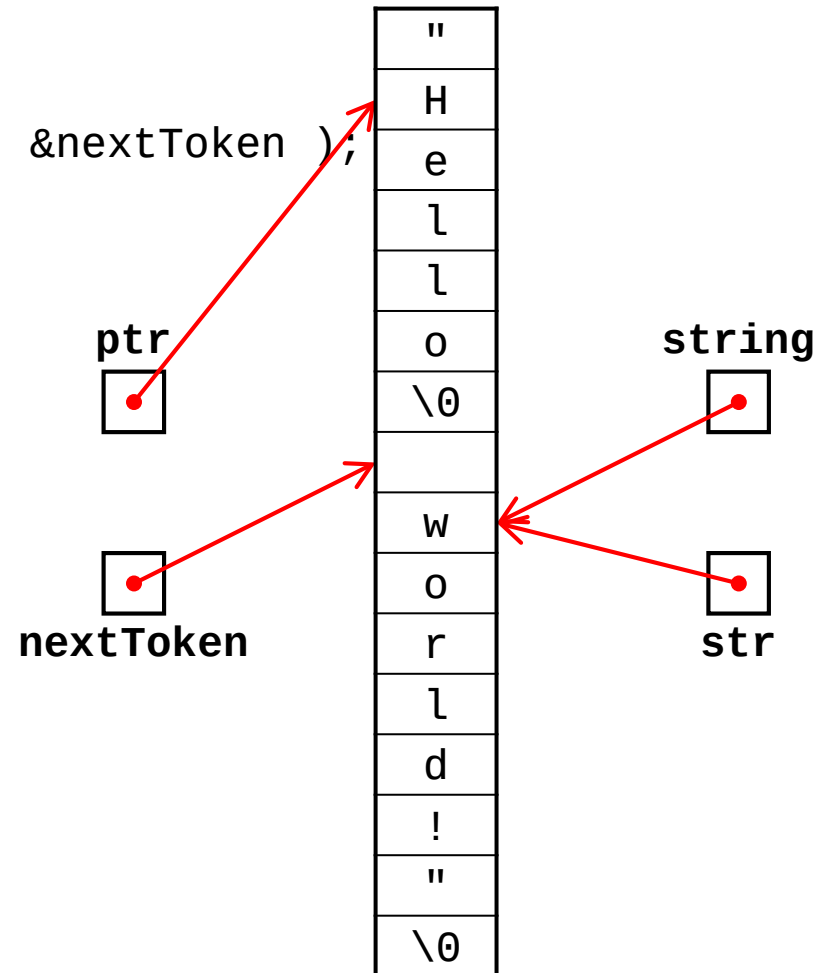
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



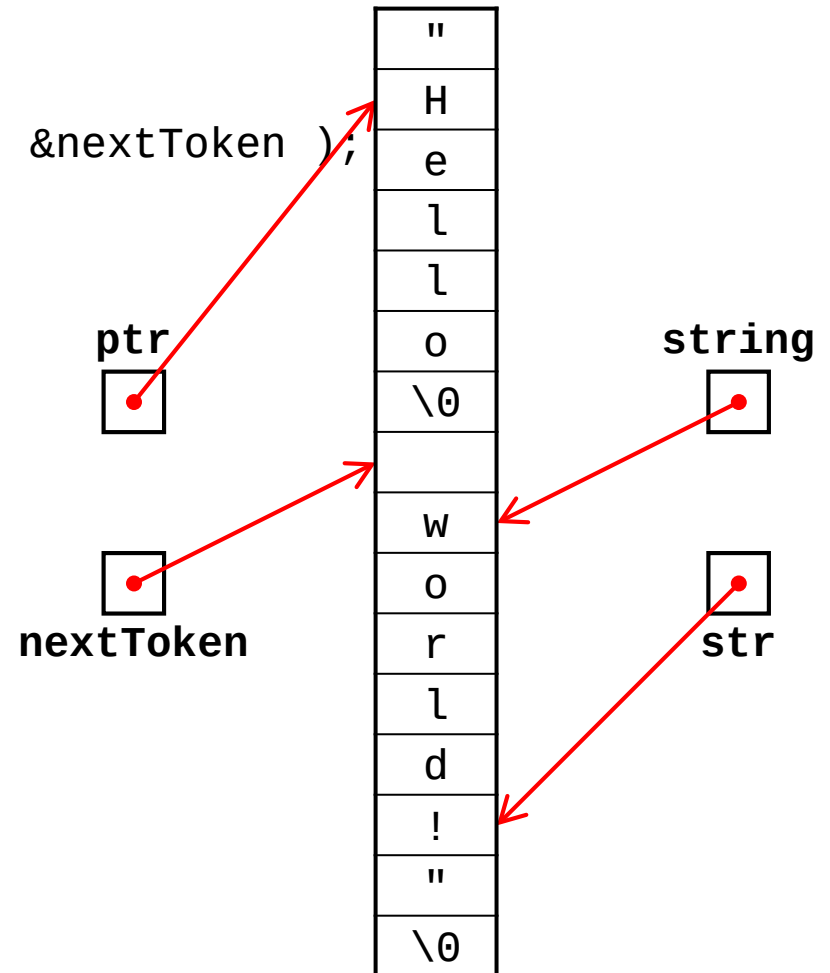
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



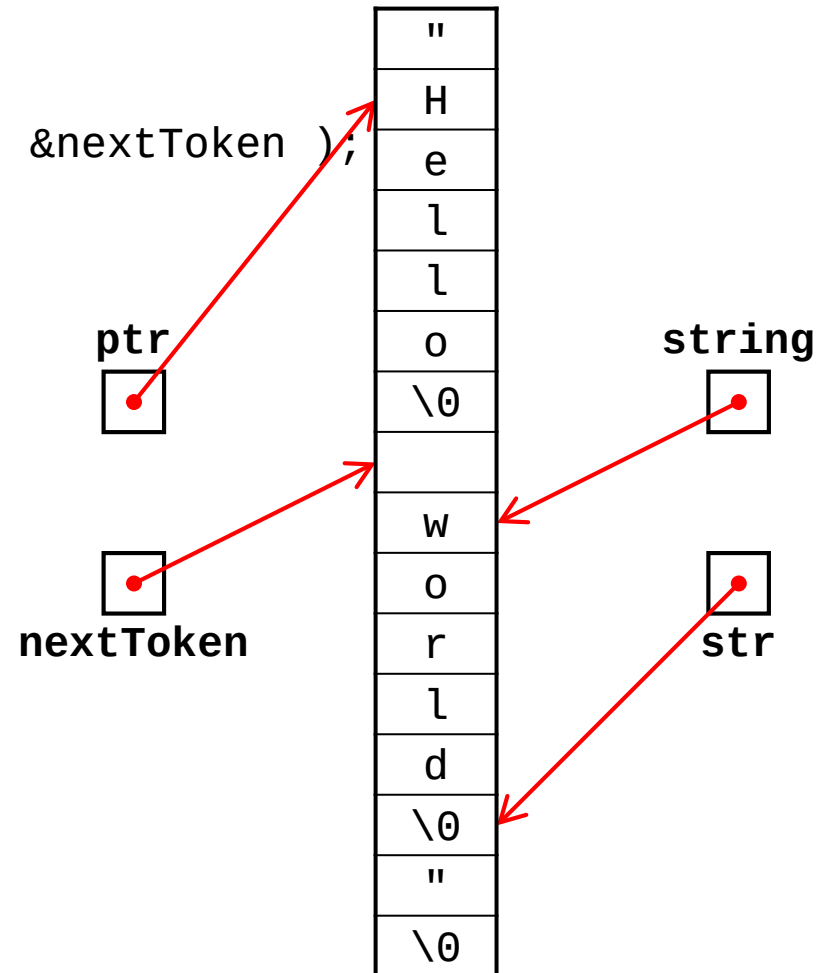
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



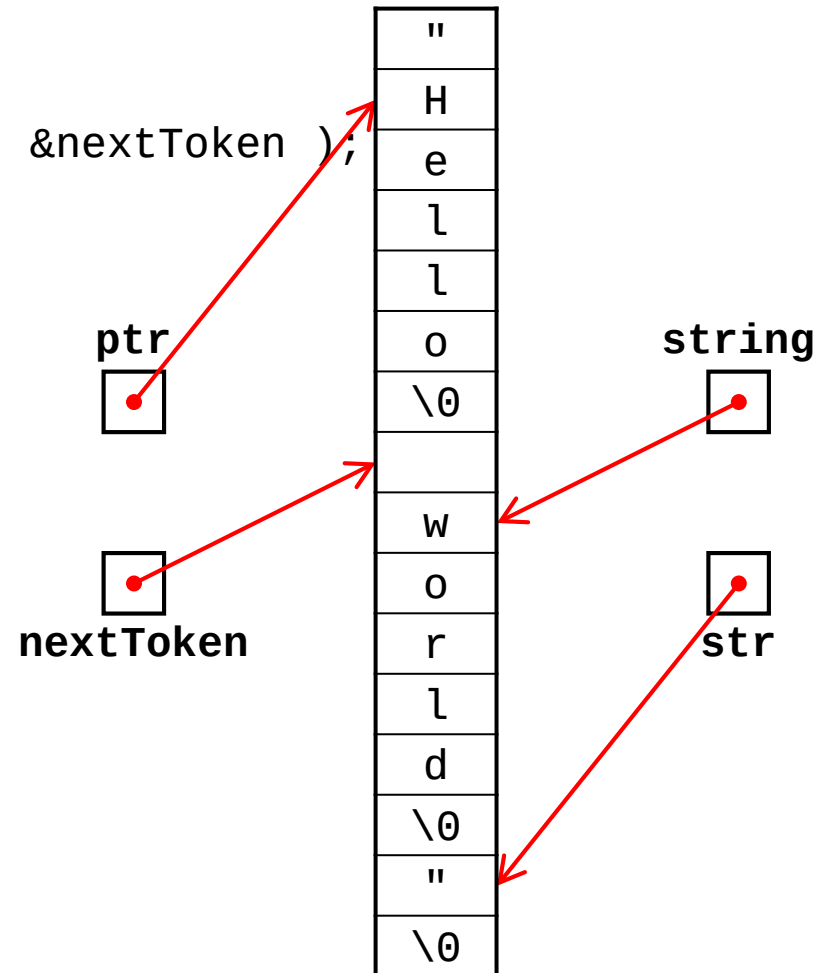
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



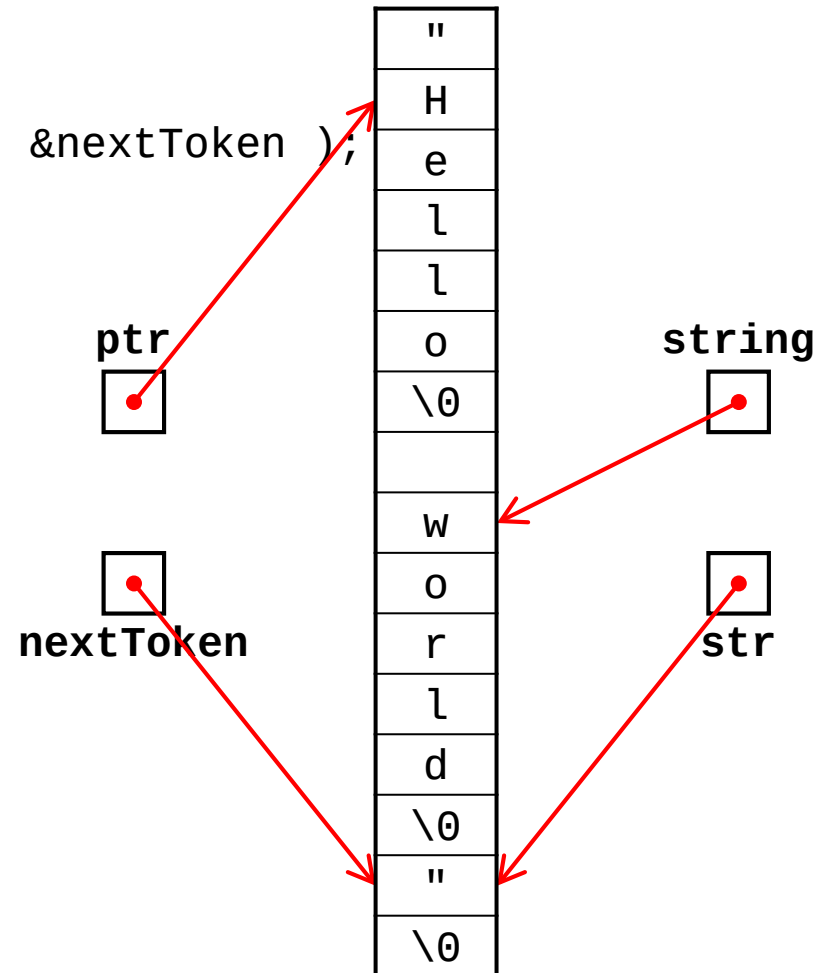
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



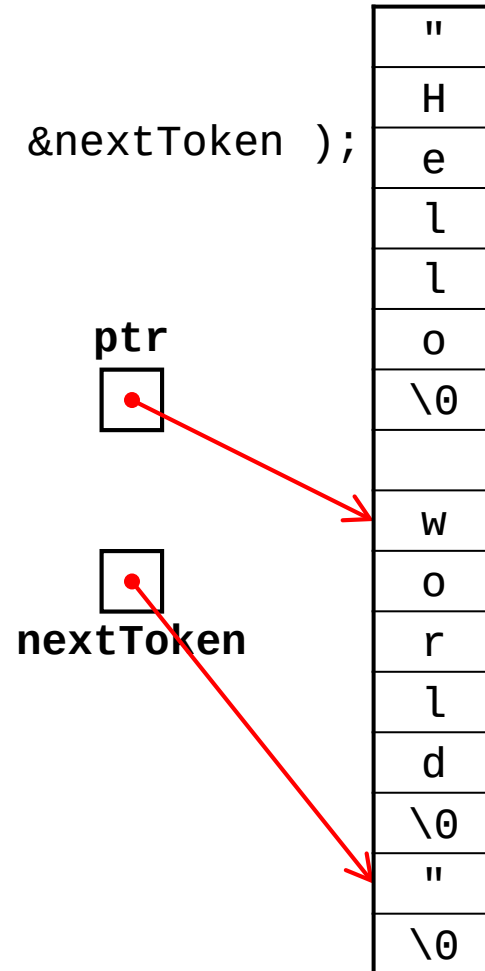
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello

Output



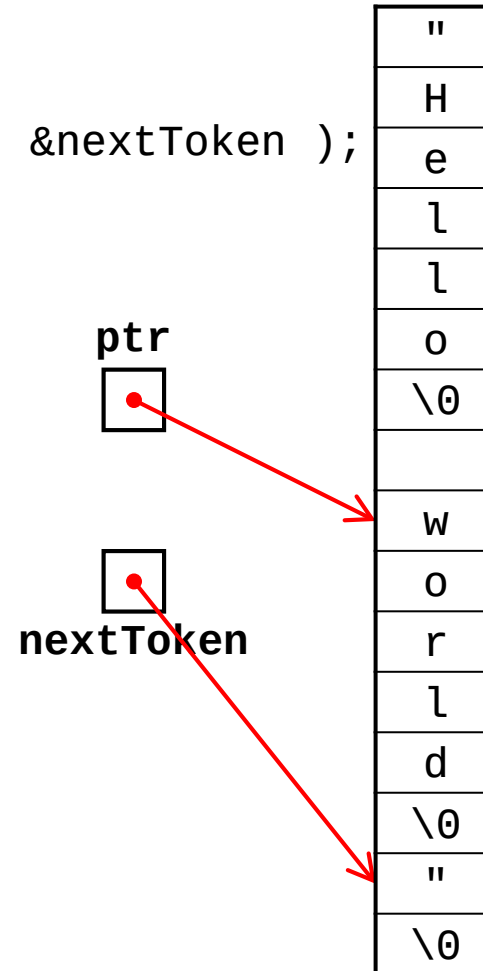
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello
world

Output



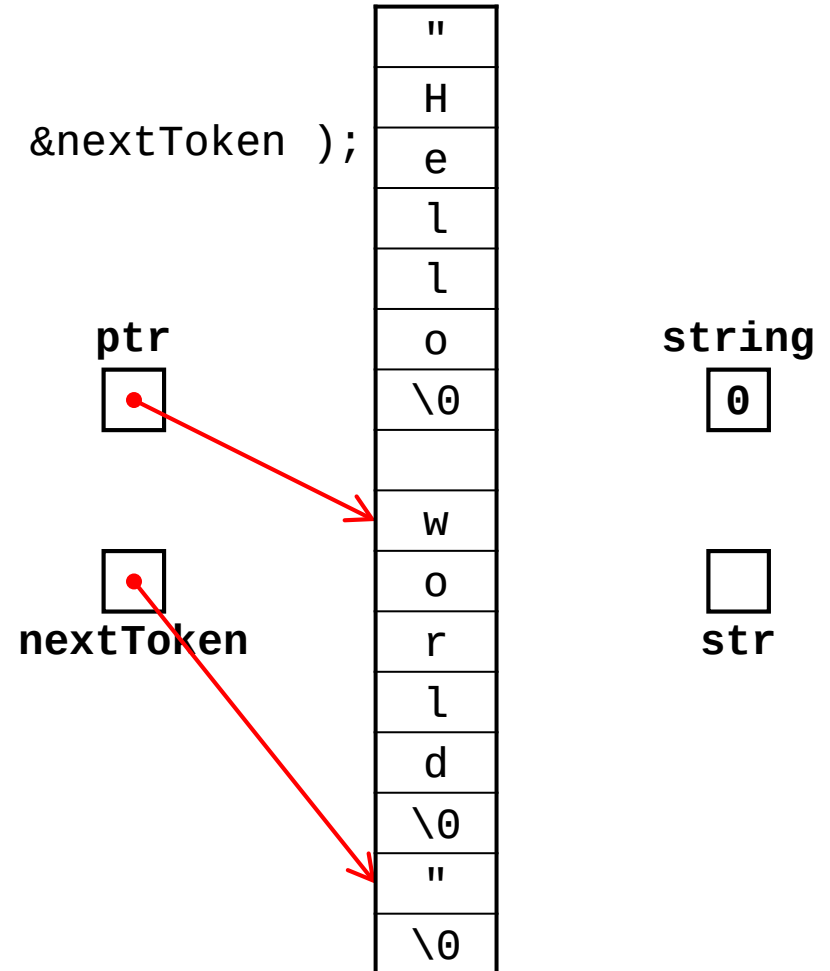

```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello
world

Output



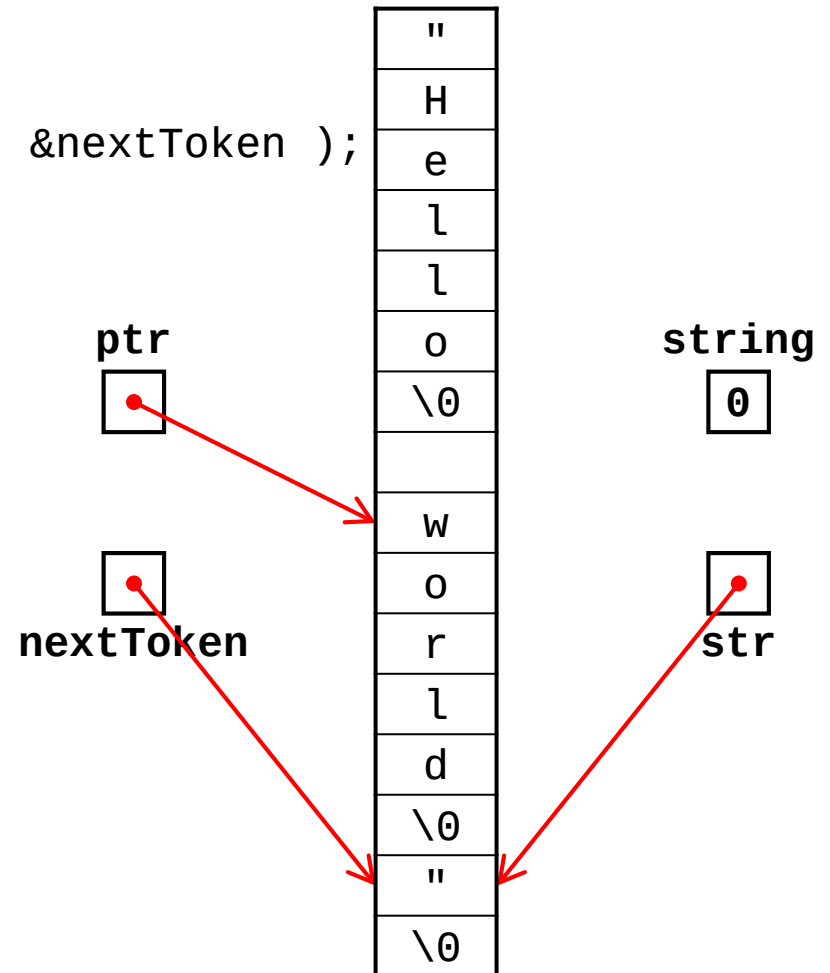
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello
world

Output



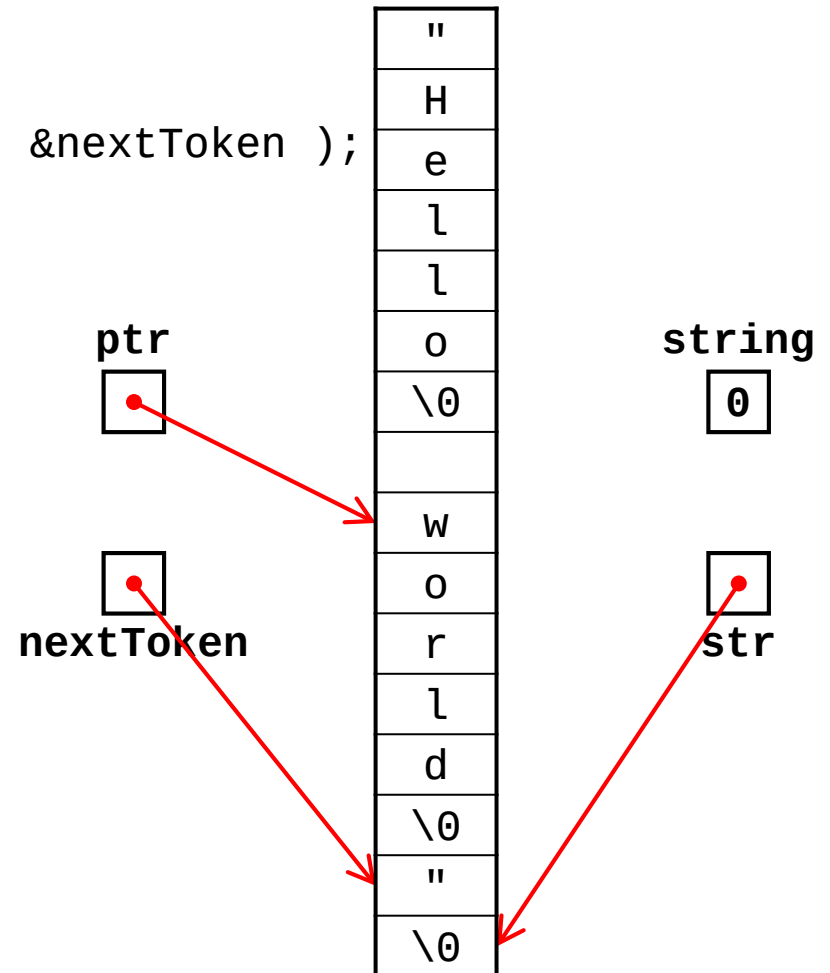
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello
world

Output



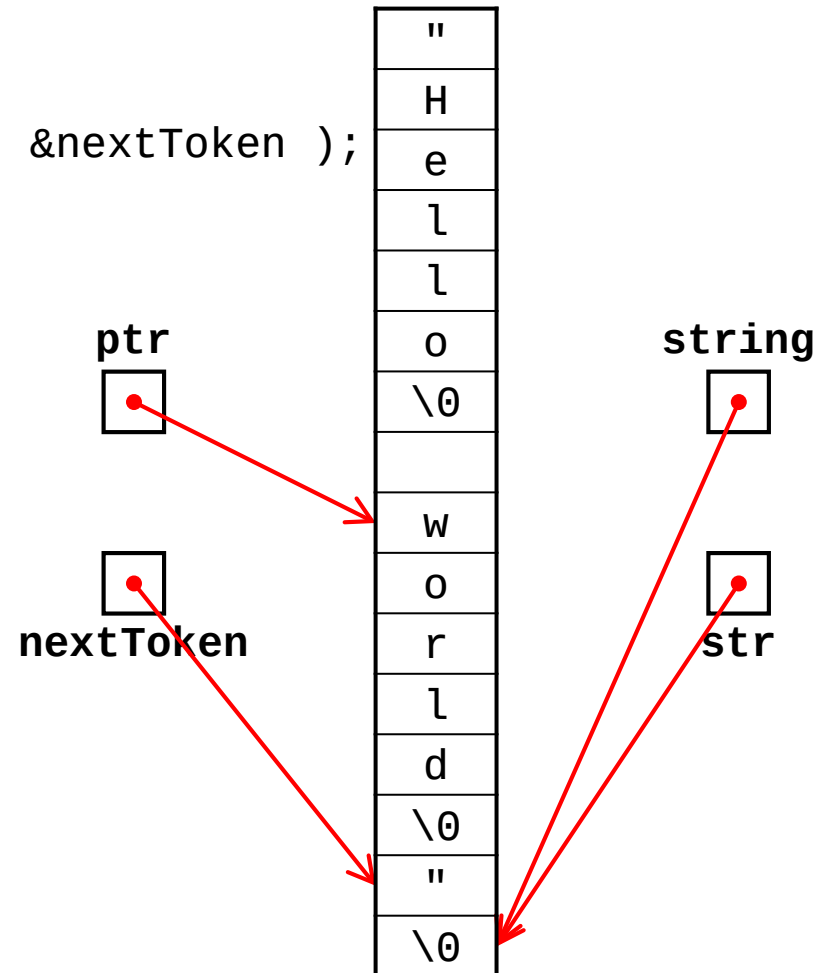
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello
world

Output



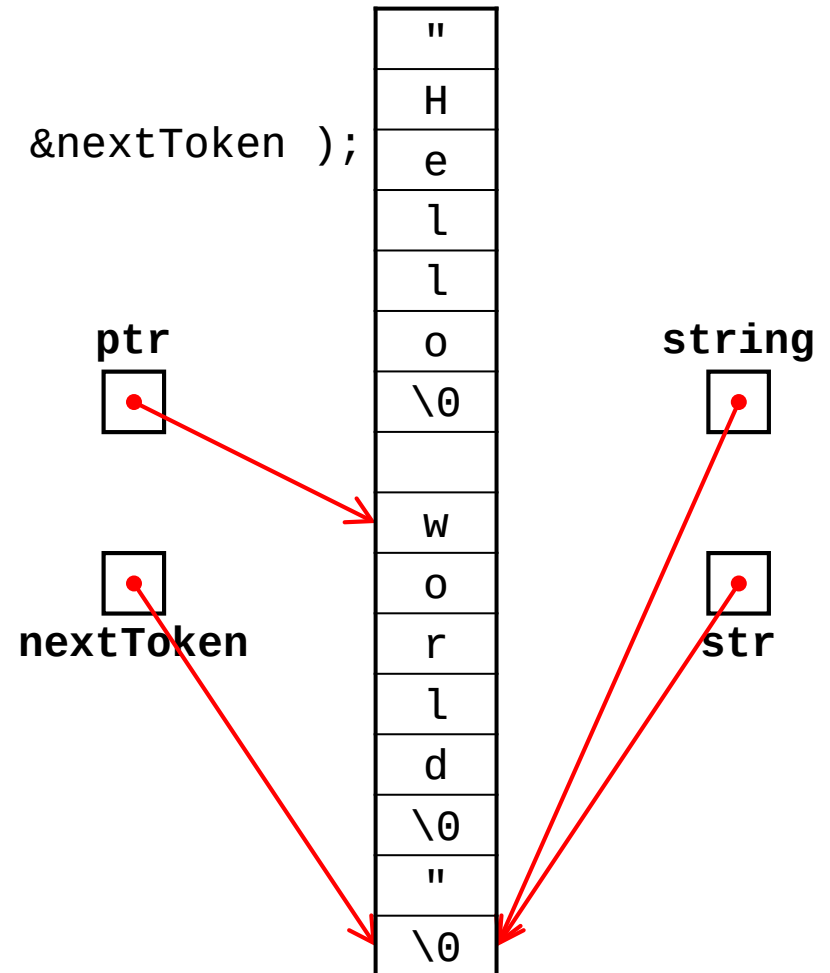
```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello
world

Output



```

int main()
{
    char string[] = "\"Hello, world!\"";
    char *nextToken;
    char *ptr;
    ptr = strtok_s( string, "\",! ", &nextToken );
    while( ptr != nullptr )
    {
        cout << ptr << '\n';
        ptr = strtok_s( nullptr, "\",! ", &nextToken );
    }
}

```

Hello
world

Output

